Skill Navora

TRACK 17

Data Engineering & Cloud

Data engineering fundamentals: roles, file formats, ingestion (batch, streaming, API, CDC), ETL and ELT, pipeline reliability, data modelling and slowly changing dimensions, warehouses, lakes and the medallion architecture, Airflow, dbt, Spark, Kafka, data quality and observability, cloud data services and cost, Docker, Kubernetes, CI/CD, architecture and system design, Linux and troubleshooting.

15 lessons | Version 2026.10 | Updated October 2026

Created by **Varun Kumar**
www.skillnavora.in

Contents

Every entry is a clickable link. The PDF also has bookmarks for each lesson and section.

01 What Data Engineering Is, Its Roles and the Data Formats It Uses

02 Ingestion: Batch, Streaming, API, File, Database and Change Data Capture

03 ETL versus ELT and Pipeline Reliability: Idempotency, Retries and Incremental Loads

04 Data Modelling: Conceptual, Logical and Physical Models, Star and Snowflake Schemas, and SCD Types 1 to 3

05 Data Warehouses, Data Marts, Data Lakes, Lakehouses and the Medallion Architecture

06 Apache Airflow: DAGs, Tasks, Scheduling and Retries

07 dbt: Models, Tests, Documentation and Lineage

08 Apache Spark and PySpark: Distributed Processing for Large Data

09 Streaming with Apache Kafka: Topics, Partitions, Consumers and Delivery Guarantees

10 Data Quality Dimensions, Observability, Catalogues, Lineage and Classification

11 Cloud Fundamentals for Data: IaaS, PaaS, SaaS, Regions, Identity and Cost

12 Containers, Kubernetes and CI/CD for Data Pipelines

13 Data Architecture and System Design for Data and AI Systems

14 Linux and the Command Line for Data Work

15 Technical Documentation, Runbooks and Production Troubleshooting for Pipelines and AI Apps

DATA ENGINEERING & CLOUD · LESSON 1

What Data Engineering Is, Its Roles and the Data Formats It Uses

Understand what data engineers build, the path data takes from source to report, and the file formats used to store and move it (CSV, JSON, XML, Parquet, Avro and ORC), with the trade-offs in size, speed and schema that decide which to use.

Gap-fill lesson · 2026 · Beginner · 120 min

By the end of this lesson you can

- Describe the data engineering role and how it differs from analytics and data science
- Trace data from a source system to a trusted reporting table
- Compare CSV, JSON, XML, Parquet, Avro and ORC on size, speed, schema and readability
- Choose a file format for a given job, with reasons

What data engineering is

Data engineering builds and runs the systems that move, store and prepare data, so that analysts, data scientists and AI applications can use it reliably. Without reliable pipelines, a dashboard shows wrong numbers and a model trains on stale data. The data engineer is responsible for that reliability.

CASE STUDY **The order that never reached the dashboard**

A sales app writes orders to a database. A nightly job should copy them to a warehouse, so that the Monday revenue dashboard is correct. One Sunday the job silently skipped a file. The dashboard showed a 9% drop that nobody could explain for two days. Data engineering exists to make that failure impossible to hide and fast to fix.

The path data takes

```
Sources (apps, databases, APIs, files)
   └─► Ingestion: copy raw data into a landing zone
         └─► Storage: keep raw copies, cleaned data and reporting tables
               └─► Transformation: clean, join, model, aggregate
                     └─► Serving: trusted tables for analysts, models and apps
                           └─► Consumers: Power BI, notebooks, AI features
```

Every arrow is a point where data can be lost, duplicated or delayed. Most of data engineering is making those arrows reliable and visible.

Roles around the data

Role	Main work	Typical tools
Data engineer	Pipelines, storage, reliability, performance	SQL, Python, Spark, Airflow, dbt, cloud services
Analytics engineer	Modelled tables and tested transformations for analysts	SQL, dbt, a warehouse
Data analyst	Questions, dashboards, reports	SQL, Excel, Power BI
Data scientist	Models and experiments	Python, statistics, ML libraries
Platform engineer	The infrastructure the others run on	Cloud, containers, infrastructure as code

In a small team one person may do several roles. The distinction matters because each role needs different guarantees: an engineer must make loads correct and repeatable; an analyst must trust the meaning of each column.

File formats

A **file format** decides how data is laid out on disk. It affects size, how fast it can be read, whether a schema is stored with it, and whether people can open it in a text editor.

Format	Type	Schema stored?	Strength	Weakness	Typical use
CSV	Text, rows	No	Universal, readable	No types, slow, large	Exports and small files
JSON	Text, nested	No (implicit)	Nested and flexible	Verbose, slow for big data	APIs and event payloads
XML	Text, nested tags	Optional	Standard in some industries	Very verbose	Legacy and exchange formats
Parquet	Binary, columnar	Yes	Small, fast for analysis	Not human-readable	Analytics storage
Avro	Binary, row-based	Yes (in the file)	Schema evolution, streaming	Less efficient for analytics	Event streams and messages
ORC	Binary, columnar	Yes	Compact, fast for analytics	Mainly used in the Hadoop world	Big-data warehouses

Columnar formats (Parquet, ORC) store each column together. An analytics query that needs two columns from a hundred reads only those two. **Row-based** formats (CSV, Avro) store each record together, which suits writing and streaming one record at a time.

Compression (such as Snappy or gzip) makes files smaller at the cost of some processing time. Columnar files compress very well because similar values sit next to each other.

See the difference in the data

The block below uses the sales file from the course. It writes the same rows as CSV and as JSON in memory and compares their sizes. The lesson on storage and modelling returns to columnar formats in detail.

Python

```python
import io
import pandas as pd

sales = pd.read_csv("daily_sales.csv", parse_dates=["date"]).head(365)

csv_text = sales.to_csv(index=False)
json_text = sales.to_json(orient="records", date_format="iso")

print("rows:", len(sales))
print("CSV characters:", len(csv_text))
print("JSON characters:", len(json_text))
print("CSV first line:", csv_text.splitlines()[0])
print("JSON first record:", json_text[:70])
```

OUTPUT

```
rows: 365
CSV characters: 5491
JSON characters: 18981
CSV first line: date,units_sold
JSON first record: [{"date":"2022-01-01T00:00:00.000","units_sold":202},
{"date":"2022-01-
```

JSON is larger here because every record repeats the field names. Columnar binary formats remove that repetition and store types, which is why they dominate analytics storage.

Choosing a format

1. **Who reads it?** If people open it in Excel, CSV is the practical choice.
2. **How is it queried?** Analytics on large data favours Parquet or ORC.
3. **Does it change over time?** Avro handles schema evolution well for event streams.
4. **Does it cross systems?** Check that each consumer supports the format and its types.

::::warn CSV loses types A CSV has no record of whether `001` is a number or a code. Leading zeros disappear, dates become text, and a comma inside a field breaks the row. Store analytical data in typed formats, and keep CSV for exchange and inspection. ::::

Roles in practice: a pipeline's owner

Every pipeline needs a named owner who is paged when it fails, who reviews its changes and who answers questions about its tables. Without an owner, a pipeline becomes a silent dependency that nobody maintains.

INTERVIEW CORNER

What is the difference between a data engineer and a data analyst?

The analyst answers questions with data that already exists in trusted tables. The engineer makes sure those tables exist, are correct, are refreshed on time and are recoverable when something breaks. The analyst asks what happened; the engineer makes sure the answer is right.

::::key What to remember - Data engineering builds the reliable path from source to trusted tables. - Know the roles: engineer, analytics engineer, analyst, scientist and platform engineer. - Choose formats by reader, query pattern, schema needs and cross-system support. - Columnar formats suit analytics; row formats suit streams; CSV suits inspection. - Every pipeline needs an owner. ::::

Quick quiz

5 questions · tap an answer to check it

1. Which format stores data column by column and suits analytics queries?

- A CSV
- B JSON
- C Parquet
- D XML

Answer: C. Columnar storage reads only the columns a query needs.

2. What is a key weakness of CSV for analytical data?

- A It is too small
- B It stores no types, so leading zeros and dates can be lost
- C It cannot be opened in Excel
- D It cannot store numbers

Answer: B. Typed formats keep the meaning of each column.

3. Which format is well suited to streams of events where the schema may change?

- **A** Avro
- **B** Plain text
- **C** A PDF
- **D** An image

Answer: A. Avro stores its schema with the data and handles evolution.

4. What does a data engineer primarily guarantee?

- **A** That every chart looks good
- **B** That pipelines load correct, repeatable and recoverable data on time
- **C** That analysts never ask questions
- **D** That models are accurate

Answer: B. Reliability of the data path is the core responsibility.

5. Why does JSON use more space than Parquet for the same rows?

- **A** JSON cannot be compressed
- **B** JSON repeats field names in every record, while Parquet stores each column once with its type
- **C** JSON stores images
- **D** Parquet does not store numbers

Answer: B. Repeated keys and text values inflate JSON.

EXERCISE

Choose formats for three jobs

For each job, choose a file format and give two reasons: (a) an export that a finance team opens in Excel each Monday; (b) clickstream events arriving continuously from a website, with new fields added every few months; (c) five years of daily sales queried for monthly totals by region.

Show solution (worked answer)

Expected choices: (a) CSV, for readability and universal support, with a note that types must be checked on import. (b) Avro, for streaming and schema evolution. (c) Parquet, columnar and compressed, for analytical queries that read few columns.

DATA ENGINEERING & CLOUD · LESSON 2

Ingestion: Batch, Streaming, API, File, Database and Change Data Capture

Move data from its source into the platform reliably. Compare batch, streaming, API pulls, file drops, database extracts and change data capture (CDC), and learn the patterns that make each one safe to rerun, including watermarks, paging and deduplication.

Gap-fill lesson · ✦ 2026 · Intermediate · ⏱ 120 min

By the end of this lesson you can

- Choose an ingestion method from freshness, volume and source capabilities
- Ingest a file and an API into a landing zone with a load record
- Use watermarks and pagination to fetch only new data
- Deduplicate events and explain what CDC captures

What ingestion is

Ingestion copies data from a source into the platform's landing zone without changing its meaning. The landing zone is the first place the data lives on your side: a bucket, a folder or a raw table. Keeping an unchanged copy makes every later step repeatable, because you can rebuild the rest from it.

CASE STUDY **Three sources, three ways in**

A retailer needs sales from its point-of-sale files (delivered as daily CSVs), its online store (a REST API with new orders) and its inventory database (changes must appear within the hour). The same platform needs three ingestion methods. Choosing wrongly means either stale data or a source overloaded by constant queries.

The five methods

Method	How it works	Freshness	Load on source	Typical use
Batch	Copy everything, or a period, at intervals	Hours to days	Periodic and heavy	Daily files, full table dumps
Streaming	Events are sent as they happen	Seconds	Low per event	Clicks, sensor data, payments
API pull	Call an endpoint, page through results	Minutes to hours	Depends on rate limits	SaaS tools, partner data
File drop	A system places files in a folder or bucket	Per drop	Low	Vendor exports, reports
Database extract	Query a table for rows changed since a point	Minutes to hours	Query load on the database	Operational tables
Change data capture (CDC)	Read the database's change log	Seconds to minutes	Very low	Keeping a warehouse close to live

Incremental pulls with a watermark

Copying everything every time is expensive. An **incremental** pull fetches only rows changed since the last successful run. The last value seen (the **watermark**, often an `updated_at` timestamp) is stored in a control table and advanced only after the load succeeds.

Python

```python
import pandas as pd
import duckdb

con = duckdb.connect()
source = pd.read_csv("daily_sales.csv", parse_dates=["date"]).rename(columns={"date":
"updated_at"})
con.register("source_rows", source)

# Control table: one row per source, holding the last watermark that loaded successfully
con.execute("CREATE TABLE IF NOT EXISTS load_control (source VARCHAR PRIMARY KEY, watermark
TIMESTAMP)")
con.execute("CREATE TABLE IF NOT EXISTS landing_sales (updated_at TIMESTAMP, units_sold
BIGINT)")

def incremental_load(con, source_name, cutoff):
    wm = con.execute("SELECT watermark FROM load_control WHERE source = ?",
[source_name]).fetchone()
    last = wm[0] if wm else pd.Timestamp("1900-01-01")
    new = con.execute("SELECT * FROM source_rows WHERE updated_at > ? AND updated_at <= ?",
                      [last, cutoff]).fetchdf()
    con.register("batch", new)
    con.execute("INSERT INTO landing_sales SELECT updated_at, units_sold FROM batch")
    new_wm = new["updated_at"].max() if len(new) else last
    con.execute("INSERT OR REPLACE INTO load_control VALUES (?, ?)", [source_name, new_wm])
    return len(new), new_wm

print(incremental_load(con, "pos_daily", pd.Timestamp("2025-01-31")))
print(incremental_load(con, "pos_daily", pd.Timestamp("2025-01-31")))   # rerun: loads
nothing new
print(con.execute("SELECT COUNT(*) FROM landing_sales").fetchone())
```

OUTPUT

```
(1127, Timestamp('2025-01-31 00:00:00'))
(0, datetime.datetime(2025, 1, 31, 0, 0))
(1127,)
```

The second run loads nothing, because the watermark already covers that period. The watermark moves only when the load succeeds, so a failed run leaves it where it was and the next run retries the same rows. The rerun safety here is the same idea as the idempotent loads in the SQL lessons.

::::warn Watermarks on event time can miss late rows A row dated yesterday that arrives today is skipped by a watermark on the event date. Use the time the record was written or updated when the source provides it, or re-load a short safety window and deduplicate. ::::

Paging through an API

APIs rarely return everything at once. They return pages, often with a cursor or a `next` link. The ingestion code must follow every page, handle rate limits, and stop only when the source says there is nothing left.

Python

```python
# Illustrative pattern (not run: needs a live API). Replace the URL and field names
# with the source's documentation.
import time
import requests

def fetch_all(url, token, page_size=100, max_retries=5):
    headers = {"Authorization": f"Bearer {token}"}
    cursor, rows = None, []
    while True:
        params = {"limit": page_size}
        if cursor:
            params["cursor"] = cursor
        for attempt in range(max_retries):
            resp = requests.get(url, headers=headers, params=params, timeout=30)
            if resp.status_code == 429:              # rate limited: wait, then retry
                time.sleep(2 ** attempt)
                continue
            resp.raise_for_status()
            break
        body = resp.json()
        rows.extend(body["data"])
        cursor = body.get("next_cursor")
        if not cursor:
            return rows                               # no more pages: stop
```

Every API has its own limits and paging style. Read the documentation before writing the loop, and record how many rows each run fetched so that a silent change in page size is visible.

Streaming and deduplication

Streaming sources can deliver the same event twice (for example, after a network retry). Deduplicate on a unique event identifier, not on the whole row, because two genuine events can look identical.

Python

```python
import pandas as pd

events = pd.DataFrame({
    "event_id": ["e1", "e2", "e2", "e3"],          # e2 was delivered twice
    "order_id": ["A10", "A11", "A11", "A12"],
    "amount":   [120.0, 80.0, 80.0, 45.0],
})
unique_events = events.drop_duplicates(subset=["event_id"], keep="last")
print(unique_events)
print("duplicates removed:", len(events) - len(unique_events))
```

OUTPUT

```
  event_id order_id  amount
0       e1      A10   120.0
2       e2      A11    80.0
3       e3      A12    45.0
duplicates removed: 1
```

Change data capture

CDC reads the database's own log of inserts, updates and deletes, instead of querying the tables. It sees every change, including deletes that a query would miss, and it puts little load on the database.

Approach	Sees deletes?	Sees every change?	Load on source
Query by updated_at	No (deleted rows vanish)	Only if the column is reliable	Query load
Full reload	Yes, by comparison	Yes, but slowly	Heavy
CDC from the log	Yes	Yes	Very low

CDC needs access to the database log and careful handling of schema changes: when a column is added, the capture must know about it before the next change arrives. Tools built for this exist; check the documentation of the database and tool you choose.

Landing zone rules

- Store the raw copy unchanged, with the load time and source name attached.
- Keep one load record per run: source, rows read, rows written, watermark, status.
- Never transform data on the way in; transform in the next layer.
- Keep raw data long enough to rebuild the layers above it.

INTERVIEW CORNER

How would you ingest data from a source that has no timestamp column?

Options: a full reload with a comparison of keys to find inserts, updates and deletes, if the table is small; CDC from the database log, if access is available; or a trigger that writes changes to an audit table. State the trade-off: full reloads are simple but heavy, CDC is precise but needs access and maintenance.

::::key What to remember - Pick ingestion by freshness, volume and what the source allows. - Incremental loads use a watermark that moves only after success. - Follow every page of an API and respect its rate limits. - Deduplicate streams on a unique event ID. - CDC captures deletes and changes with little load, but needs access and schema care. ::::

Quick quiz

5 questions · tap an answer to check it

1. When does a watermark move forward?

- **A** At the start of the run
- **B** Only after the load succeeds
- **C** Every hour regardless of success
- **D** When the source changes

Answer: B. A failed run must leave the watermark where it was.

2. Why follow the `next` cursor of an API?

- **A** To make the data bigger
- **B** Because a single response does not contain all the records
- **C** To reduce cost to zero
- **D** To change the schema

Answer: B. Paging is how APIs return large results.

3. What is the main advantage of CDC?

A It is always the cheapest to build

B It captures every change, including deletes, with little load on the source

C It requires no access

D It replaces the warehouse

Answer: B. CDC reads the change log rather than querying tables.

4. Two rows in a stream are identical except for their timestamps. What should you do?

A Remove both

B Deduplicate on the unique event ID, since they may be genuine separate events

C Keep only the first row by content

D Ignore duplicates

Answer: B. Event IDs, not row contents, identify a unique event.

5. A source has no updated_at column and is small. Which approach is simplest and safe?

A Stream it

B A full reload with key comparison to find changes

C Ignore changes

D Store only the first row

Answer: B. A small table can be reloaded and compared without special access.

EXERCISE

Ingest with a safety window

Using `daily_sales.csv`, write an incremental load that uses a 3-day safety window: each run reloads the last three days of rows and deduplicates on the date. Run it twice with the same cutoff and once with a cutoff two days later, and report the row count after each run.

Show solution (worked answer)

Hints: keep the watermark as the maximum date loaded. Deduplicate on `updated_at` after the insert, keeping the last. The row count should not change on a rerun with the same cutoff, and should grow only by the new days.

DATA ENGINEERING & CLOUD · LESSON 3

ETL versus ELT and Pipeline Reliability: Idempotency, Retries and Incremental Loads

Choose between transforming data before loading (ETL) and loading first then transforming (ELT), and build pipelines that survive reruns, retries and partial failures, using idempotent writes, bounded retries, incremental loads and reconciliation checks.

Gap-fill lesson | ✦ 2026 | Intermediate | ⏱ 120 min

By the end of this lesson you can

- Explain when ETL is the right choice and when ELT is
- Make a load idempotent with overwrite-by-partition or merge-on-key
- Apply bounded retries with backoff and know what must not be retried
- Reconcile a load against its source with counts and totals

ETL and ELT

ETL (extract, transform, load) cleans and reshapes data in a separate processing step, then loads the result. **ELT** (extract, load, transform) loads the raw data first, then transforms it inside the warehouse. The difference is where the work happens and what is kept.

Question	Favours ETL	Favours ELT
Must sensitive data be masked before it lands?	Yes	No, if the warehouse can restrict access
Is the warehouse elastic and cheap to compute?	Less relevant	Yes
Must raw data be kept for rebuilds?	Less often	Yes
Are many teams building different views of the same raw data?	No	Yes
Is the transformation complex and not SQL-friendly?	Yes	Harder

CASE STUDY **Masking before the warehouse**

A bank's card transactions include full card numbers. The warehouse is shared with many teams. The pipeline must replace the card numbers with a token before anything is written to the warehouse. That is an ETL requirement: the transformation happens before the load, so raw card numbers never land.

Most modern cloud teams use ELT for analytics and ETL for sensitive or complex steps. Both are valid; the choice is a decision about where the risk and the cost should sit.

Idempotency: the property that makes reruns safe

An operation is **idempotent** when running it again gives the same final result. Pipelines rerun all the time: after a failure, after a fix, or when a scheduler fires twice. A load that simply appends rows doubles the totals on every rerun. Two patterns make a load idempotent:

1. **Overwrite by partition.** Each run rewrites the partition it covers (for example, one day). Rerunning the same day replaces the same rows.
2. **Merge on a key.** Each row has a stable key; the load updates existing keys and inserts new ones.

Python

```python
import pandas as pd
import duckdb

con = duckdb.connect()
sales = pd.read_csv("daily_sales.csv", parse_dates=["date"])
con.register("sales", sales)
con.execute("CREATE TABLE fact_daily AS SELECT * FROM sales WHERE 1 = 0")

def load_partition(con, day):
    """Overwrite exactly one day's partition, so reruns cannot double it."""
    con.execute("DELETE FROM fact_daily WHERE date = ?", [day])
    con.execute("INSERT INTO fact_daily SELECT * FROM sales WHERE date = ?", [day])
    return con.execute("SELECT COUNT(*) FROM fact_daily WHERE date = ?", [day]).fetchone()
[0]

print("first run:", load_partition(con, "2025-01-06"))
print("rerun:    ", load_partition(con, "2025-01-06"))     # same result, not doubled
print("total rows:", con.execute("SELECT COUNT(*) FROM fact_daily").fetchone()[0])
```

OUTPUT

```
first run: 1
rerun:     1
total rows: 1
```

The rerun reports the same count, and the table holds one copy of each day. Compare that with a plain `INSERT` run twice, which would double the day.

Retries: bounded, backed off and selective

Some failures are temporary (a network timeout, a database that is briefly busy). Retrying them is correct. Other failures are permanent (a missing column, a bad credential) and retrying them wastes time and can hide the cause. A retry policy needs three rules:

- **Bounded:** a fixed number of attempts, then fail loudly.
- **Backed off:** wait longer between attempts (1, 2, 4 seconds), so the system can recover.
- **Selective:** retry only errors that can be temporary.

Python

```python
import time

def with_retries(fn, attempts=4, base_delay=0.01, retryable=(TimeoutError,
ConnectionError)):
    for i in range(attempts):
        try:
            return fn()
        except retryable:
            if i == attempts - 1:
                raise                                   # give up and surface the error
            time.sleep(base_delay * (2 ** i))
        # any other exception is not retried: it is a permanent error

calls = {"n": 0}
def flaky_load():
    calls["n"] += 1
    if calls["n"] < 3:
        raise TimeoutError("warehouse busy")
    return "loaded"

print(with_retries(flaky_load), "after", calls["n"], "attempts")
```

OUTPUT

```
loaded after 3 attempts
```

A schema error or a permission error is not in the `retryable` list, so it fails immediately with its real message. The retry is for the transient failure only.

::::warn Do not retry non-idempotent writes blindly Retrying a write that may have partly succeeded can duplicate it. Pair retries with idempotent loads (partition overwrite or merge on key), or with a check that asks whether the write already happened. ::::

Incremental loads and state

An incremental pipeline keeps a small piece of state: the watermark from the ingestion lesson, and the last successful partition. Store the state only after the data is committed, so a crash between the two leaves the state behind the data, and the next run safely reprocesses rather than skipping.

Reconciliation: prove the load is complete

A load that finishes without an error is not necessarily correct. **Reconciliation** compares the target with the source: row counts, sums of key measures, and distinct key counts, per partition.

Python

```python
src = sales[sales["date"] == "2025-01-06"]
tgt = con.execute("SELECT * FROM fact_daily WHERE date = '2025-01-06'").fetchdf()

checks = {
    "row count": (len(src), len(tgt)),
    "units total": (int(src["units_sold"].sum()), int(tgt["units_sold"].sum())),
}
for name, (expected, actual) in checks.items():
    status = "PASS" if expected == actual else "FAIL"
    print(f"{status}  {name}: source={expected} target={actual}")
```

OUTPUT

```
PASS  row count: source=1 target=1
PASS  units total: source=297 target=297
```

Run reconciliation after every load, store the results, and alert on any failure. A mismatch is the earliest and cheapest sign of a bad load.

A reliability checklist

- [ ] Each load is idempotent: a rerun gives the same result.
- [ ] Retries are bounded, backed off and limited to transient errors.
- [ ] State (watermark, last partition) advances only after a commit.
- [ ] Reconciliation runs after every load, with stored results.
- [ ] Failures alert a named owner with the partition and the error.
- [ ] A rerun procedure is written down and has been tested.

INTERVIEW CORNER **What makes a pipeline safe to rerun?**

Each run writes to a deterministic target: it overwrites the partition it owns or merges on a key, so the same input always produces the same output. Retries cover only transient errors, state advances after the commit, and reconciliation proves the result. Those four together are what make rerunning a routine action rather than a risk.

::::key What to remember - ETL transforms before loading; ELT loads raw data and transforms in the warehouse. Choose by risk, cost and rebuild needs. - Idempotency means a rerun gives the same result: overwrite partitions or merge on keys. - Retry only transient errors, with a limit and backoff. - Advance state only after commit, and reconcile every load against its source. ::::

Quick quiz

5 questions · tap an answer to check it

1. Why choose ETL for card numbers?

- **A** It is faster
- **B** The sensitive data must be masked before it lands in the shared warehouse
- **C** ELT cannot store text
- **D** It needs no testing

Answer: B. Transformation before loading protects raw sensitive data.

2. A load appends rows and is rerun after a timeout. What is the likely result?

- **A** Nothing changes
- **B** Rows are duplicated and totals are overstated
- **C** The table is deleted
- **D** The load is faster

Answer: B. Appends are not idempotent.

3. Which error should a retry policy retry?

- **A** A missing column
- **B** A wrong password
- **C** A connection timeout
- **D** A syntax error in the query

Answer: C. Only temporary failures are worth retrying.

4. What does reconciliation compare?

A The colours of the dashboard

B Counts and totals in the target against the source, per partition

C The names of the engineers

D The file sizes

Answer: B. Reconciliation proves completeness and correctness.

5. When should the watermark or last-partition state be advanced?

A Before the load starts

B After the data is committed successfully

C Every minute

D Never

Answer: B. State behind the data is safe; state ahead of the data loses rows.

EXERCISE **Make a load safe to rerun**

Write a function `load_day(con, day)` that loads one day of `daily_sales.csv` into a target table using a merge on the date (replace the day if it exists), then run it three times for two different days. Show the row count after each run and a reconciliation PASS for each day.

Show solution (worked answer)

Hints: delete the day, then insert, inside one transaction (`BEGIN` and `COMMIT`). Reconcile the units total for that day against the source file. The totals should be identical after every rerun.

DATA ENGINEERING & CLOUD · LESSON 4

Data Modelling: Conceptual, Logical and Physical Models, Star and Snowflake Schemas, and SCD Types 1 to 3

Model data at three levels, from business concepts to physical tables. Build star and snowflake schemas, and handle changing dimension values with SCD types 1, 2 and 3, including a working SCD Type 2 load that keeps history and marks the current version.

Gap-fill lesson · ✦ 2026 · Intermediate · ⏱ 150 min

By the end of this lesson you can

- Describe conceptual, logical and physical data models and what each is for
- Build a star schema with a fact table and dimensions with surrogate keys
- Compare star and snowflake designs
- Implement SCD Type 1, Type 2 and Type 3 and explain when each applies

Three levels of model

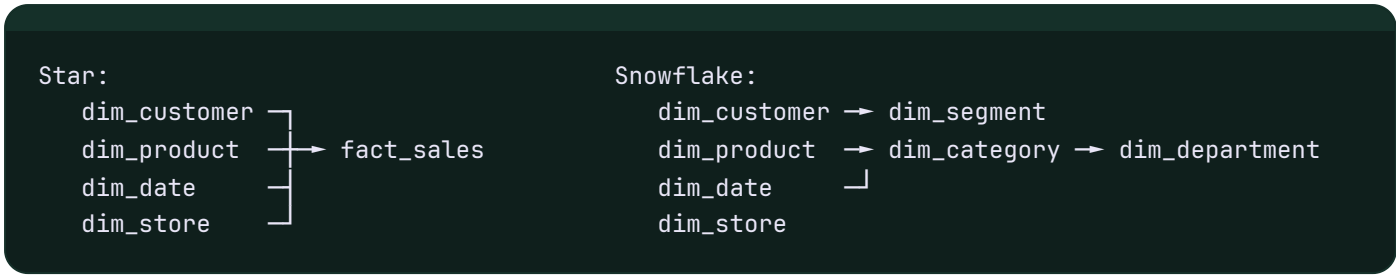
A data model describes what the data means and how it is stored. Designers work at three levels:

Level	Question it answers	Contents	Audience
Conceptual	What are the things the business cares about?	Entities (customer, order, product) and relationships	Business and analysts
Logical	What information does each entity hold?	Attributes, keys, relationships, rules, no technology	Analysts and engineers
Physical	How is it stored in this database?	Tables, types, indexes, partitions	Engineers

Start at the conceptual level with the business, move to logical, and finish at physical. A physical model built without the business concepts stores what the source happens to contain, not what the business needs.

Star and snowflake schemas

A **fact table** records measurable events (a sale, a shipment, a payment) with numeric measures and keys to the dimensions. A **dimension table** describes the context (who, what, where, when). A **star schema** keeps each dimension flat, with one table per dimension. A **snowflake schema** normalises dimensions into sub-tables.



	Star	Snowflake
Query simplicity	Simple, fewer joins	More joins
Storage	Some repetition in dimensions	Less repetition
Typical use	Reporting and BI	Very large or shared hierarchies
Maintenance	One table per dimension	Many small tables

Most reporting warehouses use stars, because the cost of repeating a few text values is small and the queries are much easier to write and understand.

Surrogate keys

A **surrogate key** is a system-generated integer with no business meaning. Facts point to surrogates, not to source keys. This keeps joins small and fast, and insulates the warehouse from source changes (a customer's email can change; the surrogate key does not).

Building a star schema

The block below builds a customer dimension with a surrogate key and a fact table pointing to it, from the course's customer and sales data.

Python

```python
import pandas as pd
import duckdb

con = duckdb.connect()
customers = pd.read_csv("customers.csv")
sales = pd.read_csv("superstore_sales.csv", parse_dates=["order_date"])
con.register("customers_src", customers)
con.register("sales_src", sales)

con.execute("""
    CREATE TABLE dim_customer AS
    SELECT ROW_NUMBER() OVER (ORDER BY customer_id) AS customer_key,
           customer_id, customer_name, segment, region
    FROM customers_src
""")
con.execute("""
    CREATE TABLE fact_sales AS
    SELECT s.row_id AS sale_id, d.customer_key, s.order_date, s.sales, s.profit, s.quantity
    FROM sales_src s
    JOIN dim_customer d ON d.customer_id = s.customer_id
""")

print(con.execute("SELECT COUNT(*) AS facts, COUNT(DISTINCT customer_key) AS customers FROM fact_sales").fetchdf())
print(con.execute("SELECT segment, ROUND(SUM(sales)) AS sales FROM fact_sales f JOIN dim_customer d USING (customer_key) GROUP BY segment ORDER BY sales DESC").fetchdf())
```

OUTPUT

```
   facts  customers
0   6323        785
       segment       sales
0     Consumer  98603292.0
1    Corporate  59113619.0
2  Home Office  43696658.0
```

The fact table holds only keys and numbers; the text lives in the dimension. A report that groups by segment joins once, on an integer.

Slowly changing dimensions

Dimensions change: a customer moves region, changes segment, or changes name. The question is what the reports should show. The answer is an **SCD type**.

Type	Behaviour	History kept?	Use when
Type 1	Overwrite the old value	No	Corrections, where the old value was wrong
Type 2	Add a new row with validity dates and a current flag	Yes, full	The business needs history as it was at the time
Type 3	Add a column for the previous value	One previous value only	Only the last change matters

SCD Type 1: overwrite

Python

```python
dim = pd.DataFrame({"customer_id": ["C1"], "region": ["North"]})
# The region was keyed wrongly: overwrite it in place, keeping no history
dim.loc[dim["customer_id"] == "C1", "region"] = "South"
print(dim)
```

OUTPUT

```
  customer_id region
0          C1  South
```

Type 1 loses the old value. Use it only when the old value was an error.

SCD Type 2: keep history

Type 2 closes the old row and inserts a new one, so that every fact joins to the version that was current on its date.

Python

```python
import pandas as pd

dim = pd.DataFrame([
    {"customer_key": 1, "customer_id": "C1", "region": "North",
     "valid_from": pd.Timestamp("2024-01-01"), "valid_to": pd.Timestamp("9999-12-31"),
"is_current": True},
])
next_key = dim["customer_key"].max() + 1

def apply_scd2(dim, customer_id, new_region, change_date):
    global next_key
    current = dim[(dim["customer_id"] == customer_id) & dim["is_current"]]
    if current.empty or current.iloc[0]["region"] == new_region:
        return dim                                      # no change: nothing to do
    idx = current.index[0]
    dim.loc[idx, "valid_to"] = change_date - pd.Timedelta(days=1)
    dim.loc[idx, "is_current"] = False
    new_row = {"customer_key": next_key, "customer_id": customer_id, "region": new_region,
               "valid_from": change_date, "valid_to": pd.Timestamp("9999-12-31"),
"is_current": True}
    next_key += 1
    return pd.concat([dim, pd.DataFrame([new_row])], ignore_index=True)

dim = apply_scd2(dim, "C1", "South", pd.Timestamp("2025-06-01"))
dim = apply_scd2(dim, "C1", "South", pd.Timestamp("2025-06-01"))   # rerun: no duplicate
version
print(dim)
```

OUTPUT

```
   customer_key customer_id region valid_from              valid_to  is_current
0             1          C1  North 2024-01-01  2025-05-31 00:00:00       False
1             2          C1  South 2025-06-01  9999-12-31 00:00:00        True
```

The rerun creates no duplicate, because the current row already has the new region. A fact dated March 2025 joins to the North version (key 1); a fact dated July 2025 joins to the South version (key 2). That is what "as it was at the time" means.

SCD Type 3: one previous value

Python

```python
dim = pd.DataFrame({"customer_id": ["C1"], "region_current": ["South"], "region_previous":
["North"]})
print(dim)
```

OUTPUT

```
  customer_id region_current region_previous
0          C1          South           North
```

Type 3 keeps a current and a previous column. It is simple and answers "what changed last" questions, but it cannot show a third region.

CASE STUDY

Which version of the customer?

A regional manager asks why last year's North sales now appear in South. The dimension was overwritten with Type 1. The fix is to use Type 2 for region, so that last year's facts keep joining to the North version, and to document the change for the team.

::::warn Overwriting history by accident Many pipelines default to Type 1 because it is easiest. If the business ever asks "what did this look like then?", Type 1 has already destroyed the answer. Decide the SCD type for each attribute with the business, and write the decision down. ::::

INTERVIEW CORNER

When would you use Type 2 rather than Type 1?

When the business needs reports to reflect the attribute as it was at the time of each event: a customer's region for sales, a product's price band for margin, an employee's department for headcount history. Type 1 is for corrections of errors, not for real changes.

::::key What to remember - Model in three levels: conceptual, logical and physical. - A star schema has a fact table and flat dimensions; a snowflake normalises them. - Use surrogate keys so facts join to stable integers. - SCD Type 1 overwrites, Type 2 keeps full history with validity dates, Type 3 keeps one previous value. - Choose the SCD type per attribute with the business. ::::

Quick quiz

5 questions · tap an answer to check it

1. What does a fact table primarily contain?

A Long text descriptions

B Measures and keys to the dimensions

C Only dates

D Passwords

Answer: B. Facts hold events' numbers and the keys that describe them.

2. Why use a surrogate key rather than the source key?

A Source keys are not allowed in databases

B It keeps joins small and insulates the warehouse from source key changes

C It removes the need for dimensions

D It makes the data larger

Answer: B. Surrogates are stable and integer-based.

3. A customer moves from North to South, and reports must show last year's sales in North. Which SCD type fits?

A Type 1

B Type 2

C No SCD is needed

D Type 0

Answer: B. Type 2 keeps the earlier version with validity dates.

4. After an SCD Type 2 change, how many rows does the customer have?

A One

B Two or more, one per version, with one marked current

C Zero

D Always one hundred

Answer: B. Each version is a row with validity dates.

5. What is the weakness of Type 3?

A It is slow

B It keeps only one previous value, so it cannot show a longer history

C It needs no keys

D It deletes data

Answer: B. Type 3 suits "last change" questions only.

EXERCISE **Add a Type 2 change to the customer dimension**

Using the Type 2 function in this lesson, record three region changes for customer C1 (North to South on 1 June 2025, South to East on 1 September 2025, East back to North on 1 December 2025). Print the dimension and then write one query that returns the region a sale dated 15 October 2025 should join to.

Show solution (worked answer)

Expected: three versions after the changes, each with its validity window, and the sale of 15 October 2025 joins to the South version. Use `valid_from ⩽ date AND date ⩽ valid_to` for the join.

DATA ENGINEERING & CLOUD · LESSON 5

Data Warehouses, Data Marts, Data Lakes, Lakehouses and the Medallion Architecture

Understand the storage choices behind a data platform: warehouse, data mart, data lake and lakehouse. Build a bronze, silver and gold (medallion) layering in code, with a quality gate between layers and a clear rule for what each layer is allowed to contain.

Gap-fill lesson | ✦ 2026 | Intermediate | ⏱ 120 min

By the end of this lesson you can

- Distinguish a data warehouse, data mart, data lake and lakehouse
- Explain the medallion architecture and what each layer holds
- Build bronze, silver and gold layers with a quality gate between them
- Decide which layer a consumer should read from

Four storage patterns

Pattern	What it holds	Strength	Weakness	Typical user
Data warehouse	Structured, modelled tables for reporting	Fast SQL, governance, trust	Costly for raw or unstructured data	Analysts and finance
Data mart	A subject-area slice of the warehouse (sales, HR)	Focused, easy to use	Can duplicate and drift if not governed	One department
Data lake	Raw files of any type on cheap storage	Cheap, flexible, keeps everything	Easy to become a swamp of untrusted files	Engineers and data scientists
Lakehouse	Open table formats on lake storage, with transactions and schemas	One copy serves analytics and machine learning	Requires careful table management	Mixed teams

A **data mart** is usually a set of tables taken from a warehouse for one team. A **data lake** is cheap storage that accepts files without a schema; without discipline it fills with duplicate and undocumented data. A **lakehouse** adds table features (transactions, schema enforcement, time travel) to lake storage, so one copy can serve both dashboards and model training.

CASE STUDY

A lake that nobody trusts

A company dumps every export into one storage bucket. Three years later, nobody knows which of the nine "sales" folders is current, and two dashboards disagree by 4%. The fix is not a bigger bucket; it is layers with rules: raw data kept as it arrived, cleaned data with checked types, and reporting tables that the business has agreed on.

The medallion architecture

The **medallion architecture** organises a lakehouse or warehouse into three layers, each with a stricter contract than the one before:

Layer	Contents	Rules	Who reads it
Bronze	Raw data as it arrived, with load time and source	No changes; append-only	Engineers (for rebuilds)
Silver	Cleaned, typed, deduplicated, conformed data	Types checked; keys unique; quality gate passed	Engineers and analysts
Gold	Business tables and metrics for reporting	Agreed definitions; documented; tested	Business users, dashboards, AI features

Each layer can be rebuilt from the one below it. If a gold table is wrong, you fix the transformation and rebuild gold; you do not patch the table by hand.

Building the layers

The block below builds bronze, silver and gold from the superstore file, with a quality gate between silver and gold. The bronze layer keeps the raw rows unchanged.

Python

```python
import pandas as pd
import duckdb

con = duckdb.connect()
raw = pd.read_csv("superstore_sales.csv", parse_dates=["order_date", "ship_date"])

# Bronze: the raw rows, unchanged, with load metadata
bronze = raw.copy()
bronze["_loaded_at"] = pd.Timestamp("2026-01-15 06:00")
bronze["_source"] = "superstore_sales.csv"
con.register("bronze_src", bronze)
con.execute("CREATE TABLE bronze_sales AS SELECT * FROM bronze_src")

# Silver: typed, deduplicated on the line key, with obvious bad rows removed
con.execute("""
    CREATE TABLE silver_sales AS
    SELECT DISTINCT ON (row_id)
           row_id, order_id, order_date, ship_date, customer_id, product_id,
           CAST(sales AS DOUBLE) AS sales, CAST(profit AS DOUBLE) AS profit,
           CAST(quantity AS INTEGER) AS quantity, region, category
    FROM bronze_sales
    WHERE sales IS NOT NULL AND quantity > 0
    ORDER BY row_id
""")

# Quality gate: the gate must pass before gold is built
gate = con.execute("""
    SELECT
      (SELECT COUNT(*) FROM bronze_sales) AS bronze_rows,
      (SELECT COUNT(*) FROM silver_sales) AS silver_rows,
      (SELECT COUNT(*) FROM silver_sales WHERE row_id IS NULL) AS null_keys,
      (SELECT COUNT(*) - COUNT(DISTINCT row_id) FROM silver_sales) AS duplicate_keys
""").fetchdf()
print(gate)
assert gate["null_keys"][0] == 0 and gate["duplicate_keys"][0] == 0, "quality gate failed"

# Gold: one agreed reporting table, monthly by region and category
con.execute("""
    CREATE TABLE gold_monthly_sales AS
    SELECT DATE_TRUNC('month', order_date) AS month, region, category,
           ROUND(SUM(sales), 2) AS sales, ROUND(SUM(profit), 2) AS profit,
           SUM(quantity) AS units
    FROM silver_sales
    GROUP BY 1, 2, 3
""")
print(con.execute("SELECT * FROM gold_monthly_sales ORDER BY month LIMIT 5").fetchdf())
```

OUTPUT

```
   bronze_rows  silver_rows  null_keys  duplicate_keys
0         6323         6323          0               0
```

```
        month region         category     sales    profit  units
0 2023-01-01  South       Technology  359670.0   8761.61   16.0
1 2023-01-01   East  Office Supplies    2265.0    424.35    9.0
2 2023-01-01  South        Furniture  253729.0  14594.30   24.0
3 2023-01-01   West  Office Supplies   80356.0  10945.12   39.0
4 2023-01-01  North  Office Supplies   16114.5   4138.33   17.0
```

Read the output of the gate: the silver row count can be lower than bronze because bad rows were removed, and the key checks must be zero. A failed assertion stops the pipeline before the gold layer is refreshed, so the reporting table is never built from bad input.

::::warn Do not skip the gate A gold table built directly from bronze is faster to write and fails silently. The gate costs one query per run and catches the errors that would otherwise appear in a board report. ::::

Which layer should a consumer read?

Consumer	Read from	Reason
Executive dashboard	Gold	Agreed definitions and tested
Analyst exploring a new question	Silver	Clean and typed, with detail
Data scientist building a model	Silver (or a feature table derived from it)	Needs detail and history
Engineer rebuilding a table	Bronze	The unchanged source
AI assistant answering business questions	Gold for metrics, silver for detail	Grounded in agreed numbers

Layer rules to write down

1. Bronze: append-only, never edited, with load time and source on every row.
2. Silver: one definition of each entity, types enforced, keys unique, and a documented list of rows removed and why.
3. Gold: each table has an owner, a description, a test on its key and a stated refresh time.
4. A table in one layer is never read by a consumer that should use another layer, unless the exception is written down.

INTERVIEW CORNER

Warehouse or lakehouse: which would you choose?

If the data is structured, the team is SQL-focused, and reporting is the main use, a warehouse is simpler and well governed. If the company also stores files, images or logs, and data scientists and analysts both need the same data, a lakehouse keeps one copy and serves both. Many organisations run a lakehouse for raw and mixed data and a warehouse or gold layer for reporting.

::::key What to remember - Warehouses hold modelled reporting data; lakes hold raw files; lakehouses add table features to lake storage. - Medallion layers: bronze raw and unchanged, silver cleaned and typed, gold agreed and tested. - Put a quality gate between silver and gold, and stop the build when it fails. - Each consumer reads from the layer that matches its trust and detail needs. ::::

Quick quiz

5 questions · tap an answer to check it

1. What does the bronze layer contain?

A Only dashboards

B Raw data as it arrived, unchanged, with load metadata

C Agreed business metrics

D Only the most recent rows

Answer: B. Bronze preserves the source for rebuilds.

2. Which layer should a board dashboard read?

A Bronze

B Any raw file

C Gold

D A personal spreadsheet

Answer: C. Gold holds agreed, tested definitions.

3. Why check for duplicate keys in silver?

A Duplicates make files larger

B Duplicate keys double-count sales in every layer above

C Duplicates are required by the medallion design

D Duplicates are removed automatically

Answer: B. Duplicates inflate totals in gold.

4. What distinguishes a lakehouse from a plain data lake?

A It has no files

B It adds table features such as transactions and schema enforcement to lake storage

C It is always cheaper

D It stores only images

Answer: B. Table formats make lake data reliable enough for analytics.

5. A gold table has a wrong total. What is the correct fix?

A Edit the gold table by hand

B Fix the transformation and rebuild gold from silver

C Delete bronze

D Ignore the difference

Answer: B. Layers rebuild from the layer below, so the fix belongs in the code.

EXERCISE **Add a second quality check**

Extend the silver layer's gate with a check that every silver `order_date` is between the earliest and latest bronze dates, and a check that `sales` is never negative. Make the build fail if either check fails, then deliberately insert one bad row into bronze and confirm the failure.

Show solution (worked answer)

Hints: count rows that break each rule and assert the counts are zero. Insert the bad row in a copy of bronze to avoid changing the raw layer; document the test.

DATA ENGINEERING & CLOUD · LESSON 6

Apache Airflow: DAGs, Tasks, Scheduling and Retries

Orchestrate a data pipeline with Apache Airflow. Define tasks and dependencies as a directed acyclic graph (DAG), schedule runs with idempotent logic, configure retries and alerts, and learn the operational habits that keep a DAG reliable.

Gap-fill lesson · ✦ 2026 · Advanced · ⏱ 120 min

By the end of this lesson you can

- Explain a DAG, its tasks and dependencies
- Write a DAG with a schedule, retries and failure alerts
- Keep tasks idempotent so that reruns and backfills are safe
- Diagnose a failed run from the task logs and the run history

What Airflow does

Apache Airflow schedules and monitors workflows. It does not process the data itself: it runs the tasks you define, in the right order, at the right time, and it records what happened. A task might call a SQL script, start a Spark job, or copy a file. Airflow decides when, in what order and what to do when a step fails.

CASE STUDY **The nightly close**

Every night a pipeline must load the day's files, run quality checks, rebuild the reporting tables and then refresh the dashboard. If the quality check fails, the dashboard must not refresh. If the load fails at 02:00, the team must know by 02:05, and the rerun must not duplicate data. An orchestrator makes each of those rules explicit.

DAGs: tasks with dependencies

A **DAG** (directed acyclic graph) is a set of tasks with arrows between them that show what must finish first. "Directed" means the arrows have a direction; "acyclic" means no task can depend on itself. The DAG is the pipeline's blueprint.

```
load_files ──► quality_check ──► build_gold ──► refresh_dashboard
                    │
                    └── (on failure) alert_owner
```

The following DAG is written for Airflow 2.x. It is **shown as code and not run**, because it needs an Airflow installation and a scheduler. Check the current Airflow documentation for the exact import paths in your version.

Python

```python
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.python import PythonOperator

default_args = {
    "owner": "data-engineering",
    "retries": 3,                          # retry transient failures
    "retry_delay": timedelta(minutes=5),
    "email_on_failure": True,
    "email": ["data-oncall@example.com"],
}

with DAG(
    dag_id="nightly_sales_close",
    start_date=datetime(2025, 1, 1),
    schedule="0 2 * * *",                  # every day at 02:00
    catchup=False,                         # do not run every missed day on deploy
    default_args=default_args,
    tags=["sales", "nightly"],
) as dag:
    load_files = BashOperator(task_id="load_files", bash_command="python load.py --date {{
ds }}")
    quality = PythonOperator(task_id="quality_check", python_callable=lambda: print("checks
passed"))
    build_gold = BashOperator(task_id="build_gold", bash_command="python build_gold.py --
date {{ ds }}")
    refresh = BashOperator(task_id="refresh_dashboard", bash_command="python refresh.py")

    load_files >> quality >> build_gold >> refresh
```

Read the key lines:

Line	Meaning
`schedule="0 2 * * *"`	A cron schedule: 02:00 every day
`retries: 3`	Retry each failing task up to three times
`retry_delay`	Wait five minutes between tries
`{{ ds }}`	Airflow's logical date, so each run processes its own day
`catchup=False`	Do not replay every past day when the DAG is first deployed
`>>`	"Must finish before": the dependency arrows

Idempotent tasks

Retries and backfills rerun tasks. A task that appends rows doubles them on every retry. Write each task so that running it for a given date twice gives the same result: overwrite the date's partition, or merge on a key. The `{{ ds }}` date in the command is what makes that possible, because the task always works on the same partition.

::::warn A task must know its date A task that uses `datetime.now()` instead of the run's logical date gives different results when it is rerun or backfilled. Pass the date in, and let Airflow supply it. ::::

Failure handling

Airflow gives several levels of response, which you should choose deliberately:

Mechanism	Use for
Retries with delay	Temporary network or database problems
Failure callback or email	Notifying the owner of a failed task
Trigger rules	Running a cleanup or alert task even when upstream fails
Sensors	Waiting for a file or a partner's data to arrive, with a timeout
SLA alerts	Warning when a task runs later than expected

A failure that should stop downstream work (a failed quality check) must block the tasks after it. A cleanup or alert task that must run regardless uses a trigger rule such as `all_done`.

Backfills

A **backfill** runs the DAG for past dates, usually after a bug fix. It is safe only if the tasks are idempotent. Run backfills in small date ranges, watch the load on the source and the warehouse, and check reconciliation after each batch.

Operating a DAG

- **Read the logs first.** The failed task's log shows the real error; the DAG graph shows where the failure stopped the chain.
- **Clear and rerun one task** after a fix, rather than rerunning the whole DAG, when the earlier tasks are already correct.
- **Keep DAGs small and readable.** A DAG with two hundred tasks is hard to debug; split it by data domain.
- **Put secrets in a connection store or environment variables,** never in the DAG file.
- **Test the DAG file** with the Airflow import check and a dry run before deploying.

INTERVIEW CORNER

How do you make an Airflow pipeline safe to rerun?

Make every task idempotent by working on the run's logical date and overwriting or merging its partition. Set bounded retries for transient errors, stop downstream tasks when a quality check fails, use `catchup=False` for new DAGs, and backfill in small batches with reconciliation after each one.

::::key What to remember - Airflow orchestrates tasks; it does not replace the tools that process the data. - A DAG is a set of tasks with dependencies; the schedule and retries are set per DAG or task. - Tasks must be idempotent and must use the run's date, not the current time. - Stop downstream work on failed checks; use trigger rules for cleanup and alerts. - Diagnose from the task log, fix, then clear and rerun the failed task. ::::

Quick quiz

5 questions · tap an answer to check it

1. What does "acyclic" mean in a DAG?

A Tasks run at random

B No task can depend, directly or indirectly, on itself

C The DAG has no schedule

D Every task runs in parallel

Answer: B. A cycle would make the order of tasks impossible to decide.

2. Why should a task use the run's logical date rather than the current time?

A The current time is always wrong

B So that a rerun or backfill works on the same partition and gives the same result

C Airflow forbids the current time

D The current time is slower

Answer: B. Using the logical date makes reruns deterministic.

3. A quality check fails. Which task behaviour is correct?

A Let the dashboard refresh anyway

B Stop the downstream tasks so that the dashboard is not refreshed with bad data

C Retry the dashboard refresh forever

D Delete the quality check

Answer: B. Downstream work must depend on passing checks.

4. What does `catchup=False` prevent?

- **A** Retries
- **B** Failure alerts
- **C** Running every missed past date when a new DAG is first deployed
- **D** Scheduling

Answer: C. It stops an unexpected flood of historical runs.

5. Which mechanism waits for a file to arrive before continuing?

- **A** A retry
- **B** A trigger rule
- **C** A sensor
- **D** A DAG tag

Answer: C. Sensors wait for a condition, with a timeout.

EXERCISE **Design a DAG**

Describe, as a diagram and a list of tasks, a DAG that loads a daily sales file, checks that the row count matches the file's own footer, builds a monthly summary, and emails the owner only when a check fails. Write the dependency line, the schedule and the retry settings, and name the one task that must be idempotent and how you would make it so.

Show solution (worked answer)

Expected: five tasks, a sensor or a file check at the start, the quality check blocking the summary, a trigger rule or failure callback for the email, a daily cron schedule, retries for transient errors, and an idempotent summary that overwrites the month's partition.

DATA ENGINEERING & CLOUD · LESSON 7

dbt: Models, Tests, Documentation and Lineage

Build transformations as tested, documented SQL models with dbt. Write models that reference each other with ref(), add schema and custom tests that run on every build, generate documentation and lineage, and organise a project so that changes are reviewed and deployed safely.

Gap-fill lesson | ✦ 2026 | Intermediate | ⏱ 120 min

By the end of this lesson you can

- Write a dbt model and reference other models with ref()
- Add generic tests (unique, not_null, accepted_values, relationships) and a custom test
- Generate documentation and read the lineage graph
- Organise a dbt project into staging, intermediate and marts layers

What dbt does

dbt (data build tool) turns SQL SELECT statements into managed tables and views, and adds the engineering practices software teams take for granted: dependencies, tests, documentation, version control and reviews. You write the transformation as SQL; dbt runs it in the right order, in your warehouse, and checks it.

CASE STUDY **A metric that changed without notice**

An analyst changed how "active customer" is calculated inside a dashboard query. Three teams reported different numbers for a month, and nobody knew which was right. In dbt, the definition lives in one model, is tested and documented, and every downstream model references it, so a change is visible in review and in the lineage graph.

Models and ref()

A **model** is a SQL file that defines one table or view. The `ref()` function points to another model; dbt uses these references to build the dependency graph, so you never hard-code the order.

SQL

```sql
-- models/staging/stg_sales.sql
select
    row_id                      as sale_id,
    order_id,
    cast(order_date as date)    as order_date,
    customer_id,
    cast(sales as numeric(12, 2))  as sales,
    cast(profit as numeric(12, 2)) as profit
from {{ source('raw', 'superstore_sales') }}
```

SQL

```sql
-- models/marts/fct_monthly_sales.sql
select
    date_trunc('month', s.order_date) as month,
    c.region,
    sum(s.sales)  as sales,
    sum(s.profit) as profit
from {{ ref('stg_sales') }} as s
join {{ ref('stg_customers') }} as c using (customer_id)
group by 1, 2
```

`source()` names raw tables that dbt does not build. `ref()` names models that dbt builds. Both create the lineage links you will see in the documentation.

Layers in a dbt project

Layer	Folder	Rule
Staging	`models/staging`	One model per source table; renaming, typing and light cleaning only
Intermediate	`models/intermediate`	Reusable joins and logic that several marts need
Marts	`models/marts`	Business tables for reporting, with tests and descriptions

Keep business logic in marts and intermediate models, not in staging. A staging model that contains a revenue definition gets copied into every report that uses it.

Tests: generic and custom

Tests are queries that return the rows that break a rule. dbt runs them after each build; a failing test fails the build.

YAML

```yaml
# models/marts/schema.yml
version: 2
models:
  - name: fct_monthly_sales
    description: "Sales and profit per month and region. One row per month and region."
    columns:
      - name: month
        tests: [not_null]
      - name: region
        tests:
          - not_null
          - accepted_values:
              values: ["North", "South", "East", "West"]
      - name: sales
        tests: [not_null]
```

Generic tests include `unique`, `not_null`, `accepted_values` and `relationships` (a foreign key exists in the parent table). A **custom test** is a SQL query that returns the failing rows, for a business rule:

SQL

```sql
-- tests/assert_sales_not_negative.sql
-- The test passes when this query returns no rows
select sale_id, sales
from {{ ref('stg_sales') }}
where sales < 0
```

::::tip Test the grain, not only the columns The most useful test on a fact table checks its grain: that one combination of keys appears once. Add a `unique` test on a surrogate key, or a test on the combination of key columns with `dbt_utils.unique_combination_of_columns` if you use that package. ::::

Documentation and lineage

Descriptions in the YAML become documentation. Running `dbt docs generate` and `dbt docs serve` produces a website with every model, column description, test and the **lineage graph**: which

sources and models feed each model, and which dashboards depend on it. Lineage answers "if I change this column, what breaks?" before the change is made.

Running and reviewing changes

Terminal

```
dbt deps          # install packages
dbt build         # run the models and their tests in dependency order
dbt build --select fct_monthly_sales+   # the model and everything downstream of it
dbt test          # run tests only
```

A safe change follows the same path as software: a branch, a pull request that shows the changed SQL and the test results, a review, and a deployment that runs `dbt build` in a staging environment before production. Changes that alter a metric should include a comparison of old and new values for the last few months.

::::warn Incremental models need care Incremental models process only new rows for speed. A change to the logic does not rewrite old rows unless you force a full refresh. After changing an incremental model's logic, run a full refresh in staging and compare the totals with the previous version. ::::

Documentation habits that pay off

- Write a one-sentence description for every mart and every key column.
- Name the grain in the description ("one row per order line").
- Record the owner of each model and the refresh schedule.
- Keep the definition of each business metric in one model, and link to it from dashboards.

INTERVIEW CORNER **What does dbt add to plain SQL scripts?**

Dependencies between models through ref(), a single build command that runs them in order, tests that run on every build, generated documentation and lineage, and a project structure that works with version control and code review. The SQL is the same; the discipline around it is what makes it maintainable.

::::key What to remember - A dbt model is a SELECT; ref() creates dependencies and lineage. - Use staging for cleaning, intermediate for reuse, and marts for business tables. - Tests return failing rows; a failing test fails the build. - Test the grain of fact tables, and write custom tests for business rules. - Review metric changes with before-and-after comparisons. ::::

Quick quiz

5 questions · tap an answer to check it

1. What does `ref('stg_sales')` do?

- **A** It copies the table into the current file
- **B** It references another model and tells dbt to build it first
- **C** It deletes the staging model
- **D** It creates a test

Answer: B. ref() creates the dependency graph.

2. What does a dbt test return when it passes?

- **A** A row count of one thousand
- **B** No rows, because the query finds no failing cases
- **C** A chart
- **D** An error

Answer: B. A test is a query for violations; zero rows means pass.

3. Which layer should hold a revenue definition used by several reports?

- **A** Raw sources
- **B** Staging
- **C** Marts or intermediate models
- **D** Dashboards only

Answer: C. Business logic belongs in shared models, not in copies.

4. What question does the lineage graph answer?

A Who is the best analyst

B If I change this model, which models and dashboards are affected?

C How long the query takes

D Which colour to use

Answer: B. Lineage shows dependencies before a change.

5. After changing an incremental model's logic, what should you do?

A Nothing; old rows update automatically

B Run a full refresh in staging and compare totals with the previous version

C Delete the model

D Skip testing

Answer: B. Incremental models keep old rows unless forced to rebuild.

EXERCISE **Add a test and a description**

Take the `fct_monthly_sales` example and add (a) a `unique` test on the combination of month and region (or a surrogate key), (b) a custom test that fails when a month has negative total sales, and (c) a description of the grain. Explain in three sentences what a failing test would mean for the dashboard.

Show solution (worked answer)

Expected: the combination test checks grain; the custom test returns rows where monthly sales are negative; a failure stops the build so the dashboard is not refreshed with the bad numbers.

DATA ENGINEERING & CLOUD · LESSON 8

Apache Spark and PySpark: Distributed Processing for Large Data

Process data too large for one machine with Apache Spark. Understand lazy evaluation, partitions and shuffles, write DataFrame transformations in PySpark, read the execution plan, and fix the common causes of slow jobs such as skew, small files and unnecessary shuffles.

Gap-fill lesson · ✦ 2026 · Advanced · ⏱ 150 min

By the end of this lesson you can

- Explain Spark's driver, executors, partitions and lazy evaluation
- Write PySpark DataFrame transformations and actions
- Identify shuffles and reduce them
- Diagnose slow jobs from the execution plan and the Spark UI

Why Spark

Pandas works well while data fits in one machine's memory. When it does not, Spark splits the data into **partitions** and processes them in parallel on a cluster of machines. Its DataFrame API looks like pandas or SQL, but the work runs across many workers.

CASE STUDY **The job that ran for six hours**

A nightly job joined three years of transactions to a product table, then aggregated. It took six hours and sometimes failed with memory errors. The fix was not a bigger cluster: it was filtering early, broadcasting the small table, and removing an unnecessary shuffle. The job then ran in twelve minutes.

The architecture

Component	Role
Driver	Plans the work, holds the program, coordinates the executors
Executors	Worker processes that run tasks on partitions and hold cached data
Partition	A chunk of the data processed as one task
Cluster manager	Allocates machines (YARN, Kubernetes, or the managed platform's own)

Lazy evaluation

Spark separates **transformations** (filter, select, join, groupBy) from **actions** (count, show, write). Transformations only build a plan. Nothing runs until an action asks for a result. Spark can then optimise the whole plan, combining steps and skipping work that is not needed.

Python

```python
# Illustrative PySpark (not run: needs a Spark installation). Shows the pattern.
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("sales_summary").getOrCreate()

sales = spark.read.option("header", True).option("inferSchema",
True).csv("superstore_sales.csv")
products = spark.read.option("header", True).csv("products.csv")

summary = (
    sales
    .filter(F.col("order_date") ≥ "2024-01-01")          # transformation: filter early
    .join(F.broadcast(products), "product_id")           # broadcast the small table
    .groupBy("category", F.date_trunc("month", "order_date").alias("month"))
    .agg(F.sum("sales").alias("sales"), F.sum("profit").alias("profit"))
)

summary.explain()                                         # show the plan, not yet
executed
summary.write.mode("overwrite").parquet("gold/monthly_category_sales")   # action: runs the
job
```

`explain()` prints the plan Spark will run. `write` is the action that starts the work. Read the plan before you tune anything.

Shuffles: the expensive step

A **shuffle** moves data between executors so that rows with the same key end up together. Joins on non-broadcast tables and `groupBy` both shuffle. Shuffles write to disk and send data over the network, which is usually the slowest part of a job.

Operation	Shuffle?	Cheaper alternative
Filter, select, map	No	–
`groupBy` with aggregation	Yes, but Spark pre-aggregates first	Keep aggregations simple
Join with a large table	Yes, both sides	Partition both tables on the join key in advance
Join with a small table	Avoidable	`broadcast(small_table)`
Repeated sorting for each step	Often	Sort once, or avoid sorting

::::tip Broadcast the small table A join with a table that fits in memory can send a copy to every executor, so the big table never has to shuffle. Use `F.broadcast()` on the small side. Spark also does this automatically below a size threshold, but explicit hints make the intent clear. ::::

Partitions: how many and how big

- Too few partitions leave executors idle; too many add scheduling overhead and tiny files.
- A common starting point is a few partitions per core, and partitions of roughly 100 to 200 MB.
- `repartition(n)` changes the number of partitions with a shuffle; `coalesce(n)` reduces it without one.

Data skew

Skew happens when one key holds most of the rows (for example, one customer id with a million orders). The task for that key runs far longer than the others, and the job waits for it. Diagnose skew in the Spark UI: one task takes much longer than its siblings. Fixes include filtering that key separately, salting the key, or aggregating in two stages.

Small files

Writing many tiny files slows every later read, because each file has overhead. Compact them with `coalesce` or the platform's compaction feature before writing the output for downstream use.

Reading the Spark UI

Tab	Look for
Jobs and stages	Which stage is slow and how many tasks it has
Stage details	Task time spread: one long task suggests skew
SQL tab	The physical plan with its shuffle and join types
Storage	Cached data and how much memory it uses

Caching

`cache()` or `persist()` keeps a DataFrame in memory when you use it more than once. Cache after expensive steps that feed several outputs, and unpersist when you finish. Caching everything wastes memory and can slow the job.

INTERVIEW CORNER

Why is a Spark job slow, and how would you find out?

Read the plan with `explain()`, open the Spark UI, and find the slowest stage. Typical causes are a large unbroadcast join, a shuffle that could be avoided by filtering earlier, skewed keys where one task runs far longer, and many small files. Fix the largest cost first and measure again.

::::key What to remember - Spark processes partitions in parallel; transformations are lazy and actions start the work. - Shuffles are the expensive step: filter early, broadcast small tables, and avoid unnecessary sorts. - Choose partitions of about 100–200 MB and avoid tiny files. - Diagnose skew in the Spark UI and fix it with separate handling or two-stage aggregation. - Read the plan and the UI before tuning. ::::

Quick quiz

5 questions · tap an answer to check it

1. Which operation starts the work in Spark?

- **A** filter
- **B** select
- **C** write (an action)
- **D** withColumn

Answer: C. Actions trigger execution; transformations only build the plan.

2. What is the main cost in a Spark job's slow step?

- **A** Writing a comment
- **B** Shuffling data between executors
- **C** Printing the schema
- **D** Reading the header row

Answer: B. Shuffles move data across the network and disk.

3. How do you avoid a shuffle when joining a large table with a small lookup table?

- **A** Repartition both tables to one partition
- **B** Broadcast the small table
- **C** Sort both tables
- **D** Cache the output

Answer: B. A broadcast join sends the small table to each executor.

4. One task in a stage takes 40 minutes while the others take 1 minute. What is the likely problem?

- **A** The cluster has too many machines
- **B** Data skew: one key holds much more data than the others
- **C** The file has a header
- **D** The output is in Parquet

Answer: B. One long task among many short ones is the signature of skew.

5. Why avoid writing thousands of tiny files?

- **A** They cannot be read
- **B** Each file adds overhead, so later reads become slow
- **C** Spark does not support files
- **D** They cost nothing

Answer: B. Compaction reduces the overhead for downstream reads.

EXERCISE **Tune a slow plan**

Write, in pseudocode or PySpark, a job that joins a 500-million-row transactions table to a 2,000-row product table and aggregates by category. List three changes you would make to reduce the cost, the expected effect of each, and how you would confirm the improvement with `explain()` and the Spark UI.

Show solution (worked answer)

Expected changes: broadcast the product table; filter to the needed date range before the join; avoid a second shuffle by aggregating once with the right keys. Confirm by comparing the plan's join type and the stage times in the UI.

DATA ENGINEERING & CLOUD · LESSON 9

Streaming with Apache Kafka: Topics, Partitions, Consumers and Delivery Guarantees

Understand event streaming with Apache Kafka. Learn topics, partitions, offsets, consumer groups and retention, the three delivery guarantees (at most once, at least once, exactly once effect), and how to build a consumer that deduplicates and commits offsets safely.

Gap-fill lesson · ✦ 2026 · Advanced · ⏱ 120 min

By the end of this lesson you can

- Explain topics, partitions, offsets and consumer groups
- Choose a delivery guarantee and design the consumer to match
- Make a consumer idempotent with deduplication on event IDs
- Describe how Kafka fits with batch pipelines and stream processing

What Kafka is

Apache Kafka is a system for recording and distributing streams of events: a click, a payment, a sensor reading, an order status change. Producers write events to a **topic**; consumers read them. Kafka keeps the events for a set time, so consumers can read at their own pace, replay history, and join the stream later.

CASE STUDY **A payment that was counted twice**

A payments service publishes "payment captured" events. A consumer that updates the ledger crashed after writing to the ledger but before recording its progress. On restart it re-read the event and counted the payment twice. The fix was not to avoid crashes; it was to make the consumer's write idempotent, so that a repeated event changes nothing.

Topics, partitions and offsets

A **topic** is a named stream. Each topic is split into **partitions**: ordered, append-only logs. Events with the same key always go to the same partition, so their order is kept for that key. Each event in a partition has an **offset**, a sequence number that a consumer uses to say where it has read up to.

```
Topic: orders
  Partition 0:  [offset 0] [offset 1] [offset 2] ...   keys A, C, A
  Partition 1:  [offset 0] [offset 1] ...              keys B, B
```

Partitions give parallelism: more partitions let more consumers read at once. Order is guaranteed only within a partition, which is why the key matters.

Consumer groups

A **consumer group** is a set of consumers that share a topic's partitions, so each partition is read by one member of the group at a time. Adding consumers to the group spreads the load. A second, separate group reads the same topic independently, which is how one stream feeds several systems (the warehouse, a fraud check, a dashboard).

Retention

Kafka keeps events for a configured time or size, whether or not consumers have read them. Retention makes replay possible: a new consumer can start from the beginning, or a consumer can rewind after a bug fix. It is not a permanent archive; long-term storage belongs in the lake.

Delivery guarantees

Guarantee	Meaning	When it happens	Consequence
At most once	An event may be lost but is never processed twice	Offsets committed before processing	Loss on failure
At least once	An event is never lost but may be processed twice	Offsets committed after processing	Duplicates on failure
Exactly once (effect)	The result is as if each event were processed once	Idempotent processing, or transactions	Requires design or transactional features

Most systems run **at least once** and make the processing idempotent. That is simpler and more robust than trying to make delivery itself exactly once.

An idempotent consumer

The consumer below is written as a pattern, not a finished program. It records each event's ID in the same transaction as the business write, so a repeated event is recognised and skipped.

Python

```
# Illustrative pattern (not run: needs a Kafka client and a database).
# Shows the logic that makes at-least-once delivery safe.

def handle(event, db):
    with db.transaction():                          # one transaction for both writes
        if db.exists("processed_events", event["event_id"]):
            return "skipped (already processed)"    # a duplicate: do nothing
        db.execute(
            "INSERT INTO ledger (order_id, amount) VALUES (%s, %s)",
            (event["order_id"], event["amount"]),
        )
        db.execute("INSERT INTO processed_events (event_id) VALUES (%s)",
(event["event_id"],))
    return "processed"

# Offsets are committed only after handle() returns, so a crash causes a re-read,
# and the duplicate is skipped by the check above.
```

The key is that the business write and the record of processing succeed or fail together. If the record is written but the offset is not committed, the event is read again and skipped. If neither is written, the event is processed on the retry.

::::warn Commit offsets after the work, not before Committing the offset before processing turns a crash into a lost event. Commit after the work is safely stored, and make the work idempotent so that the repeat is harmless. ::::

Batch and streaming together

Kafka rarely replaces the warehouse. A common design lands the stream in the lake as files (for example, hourly Parquet partitions), and the batch pipelines of the earlier lessons process those files into reporting tables. Stream processing (Kafka Streams, Flink, Spark Structured Streaming) is used when the result must be updated within seconds, such as a fraud alert or a live stock count.

Need	Choose
Reports refreshed daily	Batch from files or tables
Reports refreshed hourly	Micro-batch from landed files
Alerts within seconds	Stream processing on Kafka
Feeding several systems from one source	Kafka topic with several consumer groups

Operational checks

- Consumer lag (how far behind the newest offset each group is) is the first health metric to watch.
- Track the number of duplicates skipped; a rising number signals retries or producer problems.
- Plan partition counts ahead, because increasing them later changes which partition each key goes to.
- Protect topics with access control; events often contain personal data.

INTERVIEW CORNER

How do you guarantee that a streaming consumer does not double-count?

Assume at-least-once delivery, because that is what most systems provide. Make each write idempotent by recording the event ID in the same transaction as the business write, skip events already recorded, and commit offsets only after the work is stored. Monitor consumer lag and the rate of skipped duplicates.

::::key What to remember - A topic is split into ordered partitions; order is guaranteed only within a partition, per key. - Consumer groups share the work and let several systems read the same topic independently. - Retention enables replay; it is not a permanent archive. - Design for at-least-once delivery and make processing idempotent. - Commit offsets after the work is stored, not before. ::::

Quick quiz

5 questions · tap an answer to check it

1. Where is event order guaranteed in Kafka?

A Across the whole topic

B Within a single partition, for events with the same key

C Between topics

D Nowhere

Answer: B. Partition-level order, with keys mapping events to partitions.

2. What does a consumer group do?

A Stores the data permanently

B Shares a topic's partitions among its members so each partition is read by one at a time

C Deletes old events

D Encrypts events

Answer: B. Consumer groups divide the work.

3. Which guarantee can lose events on failure?

A At most once

B At least once

C Exactly once (effect)

D Idempotent

Answer: A. Committing before processing risks losing work on a crash.

4. **Why record the event ID in the same transaction as the business write?**

A It makes the database faster

B So the write and the record of processing succeed or fail together, which prevents double counting

C Kafka requires it

D It removes the need for offsets

Answer: B. Atomic recording makes the duplicate check reliable.

5. **Which metric is the first health check for a consumer group?**

A The number of topics

B Consumer lag

C The message colour

D The size of the cluster only

Answer: B. Lag shows how far behind a group is.

EXERCISE

Design the offset policy

A consumer updates a customer's loyalty points from a Kafka topic. Describe the delivery guarantee you would design for, the idempotency mechanism, when offsets are committed, and what happens after a crash at each of three points: before the write, after the write but before the record, and after both.

Show solution (worked answer)

Expected: at-least-once delivery with idempotent writes; the event ID record in the same transaction; commit after success. Crash before the write: the event is re-read and processed. Crash after the write but before the record: impossible if they are in one transaction, and otherwise the duplicate is detected by the existence check. After both: the offset is committed late, the event is re-read and skipped.

DATA ENGINEERING & CLOUD · LESSON 10

Data Quality Dimensions, Observability, Catalogues, Lineage and Classification

Define data quality with measurable rules across completeness, validity, uniqueness, consistency, timeliness and accuracy. Build a quality check suite that records results, monitor pipelines for freshness, volume and schema changes, and use catalogues, lineage and classification to find and protect data.

Gap-fill lesson · 2026 · Intermediate · 120 min

By the end of this lesson you can

- Define data quality dimensions and turn each into a measurable rule
- Build a quality check suite that records pass or fail and counts
- Monitor freshness, volume and schema change as observability signals
- Use a catalogue, lineage and classification to find and protect data

Quality as measurable rules

"The data is good" means nothing until it is defined. Data quality is measured along dimensions, and each dimension becomes a rule that a query can test.

Dimension	Question	Example rule
Completeness	Are required values present?	`customer_id` is never null
Validity	Do values follow the allowed format or range?	`discount` between 0 and 1
Uniqueness	Does each entity appear once?	`row_id` is unique
Consistency	Do related values agree?	every `customer_id` in sales exists in customers
Timeliness	Is the data recent enough?	the newest order is at most one day old
Accuracy	Does the data match the real world or a trusted source?	daily totals match the finance ledger

Accuracy is the hardest dimension, because it needs a trusted reference. The others can be checked from the data itself.

CASE STUDY **The report that was complete and wrong**

A dashboard showed no missing values and no duplicates, and still reported 3% less revenue than finance. The rule set had no accuracy check. A reconciliation against the ledger found that one region's file had been loaded with an old exchange rate. Completeness and validity were fine; accuracy was not measured.

A quality check suite

Each check is a query that counts failing rows. Zero means pass. Store every result, so that trends are visible and a single failure can be investigated.

Python

```python
import pandas as pd
import duckdb

con = duckdb.connect()
sales = pd.read_csv("superstore_sales.csv", parse_dates=["order_date"])
customers = pd.read_csv("customers.csv")
con.register("sales", sales)
con.register("customers", customers)

checks = {
    "completeness: customer_id is null":
        "SELECT COUNT(*) FROM sales WHERE customer_id IS NULL",
    "uniqueness: duplicate row_id":
        "SELECT COUNT(*) - COUNT(DISTINCT row_id) FROM sales",
    "validity: discount outside 0..1":
        "SELECT COUNT(*) FROM sales WHERE discount < 0 OR discount > 1",
    "validity: non-positive quantity":
        "SELECT COUNT(*) FROM sales WHERE quantity ⩽ 0",
    "consistency: sales customer not in customers":
        "SELECT COUNT(*) FROM sales s LEFT JOIN customers c USING (customer_id) WHERE
c.customer_id IS NULL",
}

results = []
for name, sql in checks.items():
    failing = con.execute(sql).fetchone()[0]
    results.append({"check": name, "failing_rows": failing, "status": "PASS" if failing ==
0 else "FAIL"})

report = pd.DataFrame(results)
print(report.to_string(index=False))
```

OUTPUT

```
                                    check  failing_rows status
           completeness: customer_id is null             0   PASS
                uniqueness: duplicate row_id             0   PASS
             validity: discount outside 0..1             0   PASS
             validity: non-positive quantity             0   PASS
consistency: sales customer not in customers             0   PASS
```

The consistency check is the one that most often fails in practice. A sales row whose customer was deleted from the customer table disappears from a join and quietly understates totals.

::::tip Make the failure useful A check that says "FAIL" is not enough. Also store the failing rows (or a sample), the check name, the date and the threshold. Then the person on call can see what broke without rerunning the job. ::::

Thresholds and severity

Not every failure stops the pipeline. Decide severity in advance:

Severity	Example	Action
Blocking	Null primary key, duplicate keys	Stop the pipeline; do not publish
Warning	0.5% of discounts out of range	Publish with a flag; alert the owner
Informational	Row count up 12% week on week	Record for trend review

Thresholds should come from the business. A 0.5% tolerance may be acceptable for marketing data and unacceptable for invoices.

Observability: knowing when something changes

Quality rules check the data you expect. **Observability** watches for the things you did not expect. Three signals cover most cases:

1. **Freshness.** When did the newest record arrive? Alert if it is older than the agreed limit.
2. **Volume.** How many rows arrived? Alert on sharp drops or spikes against a recent average.
3. **Schema.** Did a column appear, disappear or change type? Alert before the change breaks a downstream model.

Python

```python
import pandas as pd

history = pd.Series([1000, 1040, 980, 1010, 1025, 990, 1005])   # daily row counts, last week
today = 430
mean, std = history.mean(), history.std()
z = (today - mean) / std
print(f"today={today}, mean={mean:.0f}, z={z:.1f}")
print("ALERT: volume drop" if z < -3 else "volume within normal range")
```

OUTPUT

```
today=430, mean=1007, z=-28.3
ALERT: volume drop
```

A simple statistical rule such as "more than three standard deviations from the recent mean" catches most real incidents without tuning. Use a seasonal baseline for data with weekly patterns.

Catalogues, lineage and classification

A **data catalogue** is the inventory of datasets: what each one contains, who owns it, how fresh it is, and how to get access. A catalogue entry needs a description, an owner, a refresh schedule and a link to its quality results.

Lineage records where a dataset came from and what depends on it (see the dbt lesson for model-level lineage). It answers two questions: "why is this number wrong?" (trace upstream) and "who will be affected if I change this?" (trace downstream).

Classification labels columns by sensitivity: public, internal, confidential, personal. Classification drives access rules and masking. Classify at ingestion, store the label in the catalogue, and apply the masking rule in the layer that consumers read.

Column	Classification	Rule
order_id	Internal	Available to analysts
customer_name	Personal	Masked in silver and gold, restricted in bronze
sales	Internal	Available to analysts and finance
card number	Personal, high risk	Never loaded in raw form (see the ETL lesson)

INTERVIEW CORNER

How would you know a dashboard number is wrong before a business user does?

Combine rules and signals: completeness and consistency checks on the source, freshness and volume monitoring on each load, reconciliation with a trusted total where one exists, and lineage to find the upstream table if something drifts. Alert the owner when a check fails, and show a quality status next to the dashboard.

::::key What to remember - Turn each quality dimension into a measurable rule, and record every result. - Accuracy needs a trusted reference; the other dimensions can be checked from the data. - Decide severity and thresholds with the business in advance. - Observe freshness, volume and schema to catch what you did not expect. - Catalogues, lineage and classification make data findable and protected. ::::

Quick quiz

5 questions · tap an answer to check it

1. Which dimension asks whether each entity appears once?

A Validity

B Timeliness

C Uniqueness

D Accuracy

Answer: C. Uniqueness checks for duplicates of an entity's key.

2. A check returns 0 failing rows. What does that mean?

A The check did not run

B The rule passed for every row checked

C The data is accurate

D The table is empty

Answer: B. Zero failing rows is a pass, though it says nothing about accuracy.

3. Why do consistency checks often reveal understated totals?

A They add rows

B A sales row with no matching customer drops out of an inner join, so totals fall silently

C They remove duplicates

D They change currency

Answer: B. Missing parent rows make joins lose data.

4. Which observability signal would catch a source that stopped sending files?

A Schema

B Freshness

C Accuracy

D Uniqueness

Answer: B. Freshness measures how recent the newest data is.

5. Why classify columns by sensitivity?

A To change their names

B To decide who may see them and where masking must be applied

C To speed up queries

D To remove the need for a catalogue

Answer: B. Classification drives access and masking rules.

EXERCISE **Build a suite and a severity table**

For the customers and sales files, write five quality checks (one per dimension, where the dimension applies), assign each a severity, and run them. Report which fail, what you would do about each, and one accuracy check you could not run and what reference you would need.

Show solution (worked answer)

Hints: the consistency check uses a LEFT JOIN with a NULL test. For accuracy, name a reference such as a finance ledger total by month. Put the severity in a column of the report table, so the decision is visible.

DATA ENGINEERING & CLOUD · LESSON 11

Cloud Fundamentals for Data: IaaS, PaaS, SaaS, Regions, Identity and Cost

Learn the cloud concepts a data team relies on, from service models and regions to identity and access, then map the equivalent data services across AWS, Azure and Google Cloud, and control cost with tagging, storage tiers, right-sized compute and budgets.

Gap-fill lesson · 2026 · Intermediate · 120 min

By the end of this lesson you can

- Explain IaaS, PaaS and SaaS with data-platform examples
- Choose regions, identities and access rules for a data platform
- Map a data service to its AWS, Azure and Google Cloud equivalent
- Find and reduce data platform costs with tagging, tiers and budgets

The cloud service models

The cloud rents computing as services. The three main models differ in how much the provider manages for you.

Model	You manage	Provider manages	Data example
IaaS (infrastructure)	Operating system, runtime, data, applications	Physical machines, network, storage hardware	A virtual machine you install Airflow on
PaaS (platform)	Data, code and configuration	Servers, scaling, patching, runtime	A managed warehouse or a managed Spark service
SaaS (software)	Data and users	The whole application	A BI tool in the browser, a CRM

The more managed the service, the less you operate and the more you pay per unit of work. A team with few engineers usually chooses managed platforms for data, and keeps custom infrastructure for the parts that are truly special.

CASE STUDY **Build or buy the warehouse**

A five-person team considers installing a database on virtual machines. It would need patching, backups, scaling and on-call cover. A managed warehouse removes that work at a higher price per query. The decision is about where the team's time is best spent, not which option is technically superior.

Regions, availability and residency

A **region** is a geographic location with one or more data centres. An **availability zone** is an isolated data centre within a region. Choose regions for three reasons: latency to users, data residency law (for personal data, some countries require storage within their borders), and the availability of the services you need, because not every service exists in every region.

Keep the data platform in one region where possible, and replicate deliberately for disaster recovery. Cross-region copies cost money and add latency.

Identity and access

Cloud accounts use **identities** (users, groups, service accounts) and **roles or policies** that grant permissions. Four principles keep a data platform safe:

1. **Least privilege.** Grant the smallest set of permissions a job needs.
2. **Service accounts for pipelines.** A pipeline runs under its own identity, not a person's login.
3. **No long-lived keys in code.** Use the platform's identity mechanism or a secrets manager.
4. **Audit.** Every access and change to sensitive data is logged and reviewed.

A service map across the three clouds

The names differ, but the building blocks are similar. Check each provider's current documentation before you commit to a name.

Need	AWS	Azure	Google Cloud
Object storage (files, lake)	Amazon S3	Azure Data Lake Storage Gen2	Cloud Storage
Managed warehouse	Amazon Redshift	Azure Synapse Analytics	BigQuery
Managed Spark	Amazon EMR, AWS Glue	Azure Databricks, Synapse Spark	Dataproc
Streaming	Amazon Kinesis, MSK (Kafka)	Event Hubs, Event Hubs for Kafka	Pub/Sub, Managed Kafka
Orchestration	Amazon MWAA (managed Airflow)	Azure Data Factory, Managed Airflow	Cloud Composer (managed Airflow)
Serverless functions	AWS Lambda	Azure Functions	Cloud Functions
Identity	IAM	Microsoft Entra ID, RBAC	Cloud IAM

Many organisations also use **Databricks** and **Snowflake**, which run on all three clouds, and **Microsoft Fabric** in the Microsoft ecosystem (covered in the Power BI lessons).

Cost: where the money goes

Data platform costs usually come from four places:

Cost driver	Why it grows	Control
Compute	Clusters left running; oversized machines	Auto-suspend, auto-scale, right-size
Storage	Raw copies kept forever; duplicate tables	Lifecycle rules, storage tiers, delete unused copies
Data transfer	Moving data between regions or to the internet	Keep compute near storage; avoid unnecessary copies
Queries	Scanning full tables; repeated identical queries	Partition and cluster tables; cache results; select only needed columns

Cost controls that work

1. **Tag everything** with team, product and environment, so that each cost can be assigned to an owner.

2. **Set budgets and alerts** at 50%, 80% and 100% of the monthly budget.
3. **Use storage tiers.** Move files that are rarely read to cheaper cold tiers, and delete temporary data on a schedule.
4. **Auto-suspend compute** when idle, and size clusters for the workload rather than the peak.
5. **Review monthly.** Find the top five costs, and ask who owns each and whether it is still needed.

Python

```python
# A simple monthly cost model: compare two storage choices for the same data
data_tb = 5
hot_price_per_tb = 23.0          # illustrative monthly price per TB; check the provider's price list
cold_price_per_tb = 4.0          # illustrative
share_read_monthly = 0.10        # 10% of the data is read each month

hot_all = data_tb * hot_price_per_tb
tiered = data_tb * (share_read_monthly * hot_price_per_tb + (1 - share_read_monthly) * cold_price_per_tb)
print(f"all hot: {hot_all:.2f}   tiered: {tiered:.2f}   saving: {hot_all - tiered:.2f} per month")
```

OUTPUT

```
all hot: 115.00   tiered: 29.50   saving: 85.50 per month
```

The prices are placeholders that show the method. Replace them with the current figures from the provider's pricing page, and note the date you checked.

::::warn Free tiers and surprise bills Cloud free tiers are real but limited. A query that scans a large table, a cluster left running over a weekend, or a copy sent across regions can cost more in one day than a month of storage. Set budgets before you start building. ::::

Security basics for data in the cloud

- Encrypt data at rest and in transit by default.
- Keep storage buckets private; never make a data bucket public for convenience.
- Restrict network access so that databases accept connections only from known services.
- Monitor for unusual access, and rotate credentials.

INTERVIEW CORNER

How would you reduce a data platform's cloud cost by a third?

Start with measurement: tag costs by owner and find the largest drivers. Then right-size and auto-suspend compute, move cold data to cheaper tiers, remove unused copies, partition tables so queries scan less, and set budgets with alerts. Measure after each change, and report the saving with the date.

::::key What to remember - IaaS, PaaS and SaaS differ in what you manage; managed services trade price for operational effort. - Choose regions for latency, residency and service availability. - Use service identities with least privilege and no long-lived keys in code. - The service names differ across AWS, Azure and Google Cloud; the building blocks are similar. - Control cost through tagging, budgets, tiers, auto-suspend and query design. ::::

Quick quiz

5 questions · tap an answer to check it

1. In PaaS, what does the provider typically manage that you do not?

A Your data

B Servers, scaling and patching of the platform

C Your code

D Your users

Answer: B. PaaS removes the infrastructure work from you.

2. Why use a service account for a pipeline?

A It is faster

B It gives the pipeline its own least-privilege identity instead of a person's login

C It avoids billing

D It removes the need for permissions

Answer: B. Service identities are auditable and limited.

3. Which cost driver is most often caused by a cluster left running at the weekend?

A Storage

B Data transfer

C Compute

D Documentation

Answer: C. Compute is billed while it is running.

4. What is a sensible first step in cost reduction?

A Delete all data

B Tag costs by owner and find the largest drivers

C Buy a bigger machine

D Turn off monitoring

Answer: B. Measurement shows where the money goes.

5. Which AWS service is the managed equivalent of a data warehouse?

A Amazon S3

B Amazon Redshift

C AWS Lambda

D Amazon Kinesis

Answer: B. Redshift is the managed warehouse; S3 is storage.

EXERCISE

Map and cost one pipeline

Choose a simple pipeline (files land daily, a warehouse table is built, a dashboard reads it). Map each component to a service on one cloud, list the identity each needs, and estimate the monthly cost of storage and compute using the provider's current price list. State the date you checked the prices and one cost control you would apply.

Show solution (worked answer)

Hints: count storage as the total size times the price per unit, and compute as the hours the cluster runs times its hourly price. Put placeholder prices in your working and replace them with the official figures before reporting.

DATA ENGINEERING & CLOUD · LESSON 12

Containers, Kubernetes and CI/CD for Data Pipelines

Package a data pipeline so that it runs the same way on any machine. Write a Dockerfile, understand what Kubernetes adds for scheduling and scaling, and build a continuous integration and deployment pipeline that tests a dbt or Python project before it reaches production.

Gap-fill lesson · ✦ 2026 · Advanced · ⏱ 120 min

By the end of this lesson you can

- Explain images, containers and why they make pipelines portable
- Write a Dockerfile for a Python data job and run it with mounted data
- Describe Kubernetes objects and when they are needed for data work
- Design a CI/CD pipeline that runs tests and deploys a data project safely

Why containers

A pipeline that works on one laptop often fails on a server: a different Python version, a missing library, a path that does not exist. A **container** packages the code and its dependencies into a single unit that runs the same way everywhere. An **image** is the packaged recipe; a **container** is a running copy of it.

CASE STUDY **"It works on my machine"**

A data job needed a specific pandas version and a system library. Each engineer installed them differently, and the nightly server had a third setup. Once the job ran in a container built from one Dockerfile, every environment behaved the same, and the nightly failures stopped.

A Dockerfile for a data job

A **Dockerfile** lists the steps that build an image. The example below packages a Python job that reads a CSV and writes a summary. It is shown as code and **not run**, because it needs Docker

installed.

DOCKERFILE

```
# Dockerfile
FROM python:3.12-slim

WORKDIR /app

# Install dependencies first: this layer is cached until requirements.txt changes
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy the code last: code changes do not reinstall dependencies
COPY summarise_sales.py .

# Data is mounted at run time, not copied into the image
ENTRYPOINT ["python", "summarise_sales.py"]
```

Build and run it:

Terminal

```
docker build -t sales-summary:1.0 .
docker run --rm -v "$PWD/data:/data" sales-summary:1.0 --input /data/daily_sales.csv --
output /data/summary.csv
```

Key ideas:

- **Layer order** matters. Put the things that change least (dependencies) first, so rebuilds are fast.
- **Pin versions** in `requirements.txt` so that a rebuild next month does not change behaviour.
- **Do not copy secrets into the image.** Pass them at run time as environment variables or from a secrets store.
- **Mount data** rather than baking it in. The same image can then process today's file or last month's.

::::warn Images are not secret vaults Anything written into an image layer can be read by anyone who can pull the image, even if a later layer deletes it. Keep credentials out of the Dockerfile and out of the build context. ::::

What Kubernetes adds

Kubernetes runs containers across a cluster of machines. You describe the desired state (for example, "run three copies of this worker, restart it if it crashes, give it 2 GB of memory"), and Kubernetes keeps reality matching that description.

Kubernetes object	Purpose in data work
Pod	One or more containers that run together
Deployment	Keeps a long-running service (for example a streaming consumer) alive
Job / CronJob	Runs a batch task to completion, or on a schedule
ConfigMap and Secret	Configuration and credentials, kept out of the image
Horizontal autoscaler	Adds workers when a queue grows

YAML

```yaml
# A scheduled batch job (CronJob), shown as configuration; not run
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-sales-summary
spec:
  schedule: "0 2 * * *"           # 02:00 every day
  concurrencyPolicy: Forbid       # never start a second run while one is running
  jobTemplate:
    spec:
      backoffLimit: 3             # retry up to three times
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: summary
              image: registry.example.com/sales-summary:1.0
              envFrom:
                - secretRef:
                    name: warehouse-credentials
```

Kubernetes is powerful and has a learning cost. For a single nightly job, a managed scheduler (for example Airflow or a cloud scheduler) may be simpler. Use Kubernetes when you run many services or need fine control over scaling and placement.

CI/CD for data projects

Continuous integration (CI) runs the tests and checks on every change. **Continuous deployment (CD)** releases a tested version to an environment automatically or with an approval step. For data, the pipeline should also run the data tests, not only the code tests.

A sensible pipeline for a dbt or Python data project:

YAML

```yaml
# Illustrative CI workflow (GitHub Actions style); adapt to your platform
name: data-ci
on:
  pull_request:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.12"}
      - run: pip install -r requirements.txt
      - run: python -m pytest tests/            # unit tests for transformation code
      - run: dbt build --target ci              # models and their data tests in a CI
schema
  deploy-staging:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - run: dbt build --target staging
```

The `dbt build --target ci` step builds into a temporary schema, so the tests run on real data structures without touching production. Deploy to staging only after the tests pass, and promote to production with an approval.

::::tip Test data, not only code A passing unit test does not prove the table is right. Include data tests (uniqueness, not-null, reconciliation totals) in the CI run, and fail the pipeline when they fail. A change that breaks a metric should be caught before the merge. ::::

Release habits

- Tag each release with a version number; record which version produced each output table.
- Keep environments separate (CI, staging, production) with their own credentials.
- Make deployments reversible: keep the previous image and the previous dbt state, so a rollback is one command.
- Never deploy by editing production by hand; the pipeline is the only route.

INTERVIEW CORNER

How would you ship a change to a data pipeline safely?

Work on a branch, run unit and data tests in CI against a temporary schema, compare the output metrics with the previous version, deploy to staging, then promote to production with an approval and a recorded version. Keep the previous version available so that a rollback is quick.

::::key What to remember - Containers package code and dependencies so that a pipeline runs the same everywhere. - Layer Dockerfiles for fast rebuilds, pin versions, and keep secrets out of images. - Kubernetes runs and scales containers; use it when the operational need justifies it. - CI runs unit and data tests on every change; CD promotes tested versions with approval. - Version every release and keep rollbacks simple. ::::

Quick quiz

5 questions · tap an answer to check it

1. What is the difference between an image and a container?

- A They are the same thing
- B An image is the packaged recipe; a container is a running copy of it
- C A container is a file on disk only
- D An image runs without a host

Answer: B. The image is built once; containers are started from it.

2. Why put the dependency installation before copying the code in a Dockerfile?

- A It is required by Docker
- B So that the dependency layer is cached and rebuilds after code changes are fast
- C It makes the image smaller
- D It installs the code twice

Answer: B. Layer order determines what Docker can reuse.

3. Why should secrets not be baked into an image?

- **A** They are too large
- **B** Anyone who can pull the image can read them, even if a later layer deletes them
- **C** Images cannot run with secrets
- **D** Secrets are always public

Answer: B. Image layers are readable by anyone with access.

4. What does `concurrencyPolicy: Forbid` do in a CronJob?

- **A** It stops the job from running at all
- **B** It prevents a new run starting while the previous run is still active
- **C** It doubles the retries
- **D** It removes the schedule

Answer: B. It stops overlapping runs.

5. What makes a CI pipeline valuable for data projects?

- **A** It only checks the code style
- **B** It runs data tests on a temporary schema before a change reaches production
- **C** It deploys without review
- **D** It runs once a year

Answer: B. Data tests catch errors that code tests cannot see.

EXERCISE

Write a CI plan

For a data project you know, write a CI and CD plan with five steps: the trigger, the code tests, the data tests and where they run, the staging deployment, and the production promotion with its approval. Name the one control that would have stopped the most serious past failure in that project.

Show solution (worked answer)

Hints: keep credentials in the CI system's secrets store. For the data tests, choose the three most important checks for the project. The control should be a test or an approval that the failure would have hit.

DATA ENGINEERING & CLOUD · LESSON 13

Data Architecture and System Design for Data and AI Systems

Design a data platform end to end. Describe sources, ingestion, storage, serving and governance as layers, reason about the requirements that drive each choice (freshness, scale, cost, trust, security), and apply system design basics such as bottlenecks, caching, replication and failure modes to data and AI systems.

Gap-fill lesson · ✦ 2026 · Advanced · ⏱ 150 min

By the end of this lesson you can

- Describe a data architecture with sources, ingestion, storage, serving and governance layers
- Turn business requirements into freshness, scale, cost and trust targets
- Apply system design basics to find bottlenecks and single points of failure
- Write a short architecture decision record for a key choice

Architecture starts with requirements

A data architecture is a set of decisions about how data moves and where it is stored. The decisions follow from requirements, and most bad architectures come from choosing tools before stating what they must do. Before drawing any diagram, write the requirements in measurable terms:

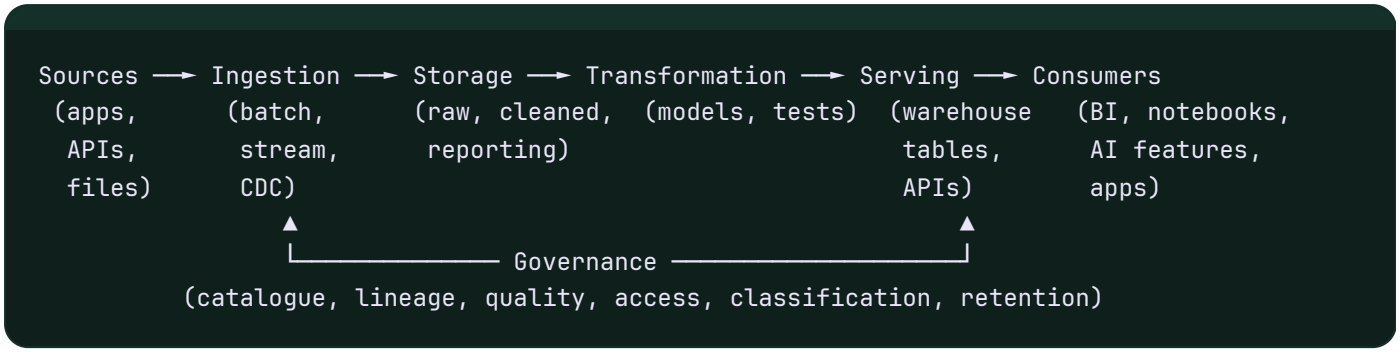
Requirement	Measurable form
Freshness	Dashboard shows data no older than 6 hours
Scale	200 million events a month, growing 30% a year
Trust	Revenue matches the ledger within 0.5% every month
Cost	Platform under £8,000 per month in year one
Security	Personal data visible only to the customer team
Availability	Reports available 99.5% of working hours

These numbers decide the architecture. A 6-hour freshness target does not need streaming; a 30-second fraud alert does.

CASE STUDY **The architecture that was too fast**

A team designed a streaming platform for a sales dashboard that the executives checked once a week. The platform cost four times the budget and was harder to test. A requirements review found that a daily batch pipeline met every need. The lesson is that the fastest architecture is not automatically the right one.

The layers of a data architecture



Layer	Question it answers	Typical choices
Sources	Where does the data originate?	Operational databases, SaaS APIs, event streams, files
Ingestion	How does it arrive?	Batch extracts, CDC, streaming, file drops
Storage	Where does it live, raw and modelled?	Object storage with open table formats; a warehouse
Transformation	How is it cleaned and modelled?	SQL models with tests; Spark for large jobs
Serving	How do consumers read it?	Warehouse tables, semantic models, APIs
Governance	Who may use it, and can we trust it?	Catalogue, lineage, quality checks, access control

Governance is drawn across the whole diagram because it is not a single step. Every layer needs ownership, quality rules and access rules.

System design basics for data

Data systems fail in the same ways as other distributed systems. Four ideas explain most of the design choices.

Bottlenecks. The slowest component sets the speed of the whole system. Find it by measuring each stage: the source's query capacity, the network, the warehouse's concurrency, the dashboard's query count. Improving anything other than the bottleneck does nothing.

Caching. Store the result of an expensive computation and reuse it. A reporting table that is rebuilt nightly is a cache of the detailed data. A cache is correct only when its refresh rule matches the freshness requirement, and stale caches are a classic source of wrong numbers.

Replication and backup. Copies protect against loss and let reads spread out. Backups protect against mistakes: a deleted table or a bad load. Replication alone copies the mistake; backups and time travel let you go back.

Failure modes. Ask what happens when each component fails: the source is down, the ingestion job crashes, the warehouse is slow, a schema changes upstream. A good design says, for each failure, whether data is delayed, lost, duplicated or served wrongly, and how the team will know.

Failure	Possible effect	Design response
Source unavailable	Data delayed	Retry; alert on freshness; show "last updated" on dashboards
Duplicate delivery	Totals overstated	Idempotent loads and deduplication
Schema change upstream	Pipeline breaks or silently mis-reads	Schema checks; contracts with the source team
Warehouse overloaded	Dashboards slow	Serve summaries; limit concurrency
Bad load publishes	Wrong numbers visible	Quality gate before publish; rollback

A design walk-through: a retail sales platform

Requirements. Dashboards refresh by 07:00 with yesterday's sales (batch is enough). Revenue must reconcile with the ledger monthly. Personal customer data visible only to the customer team. Budget modest.

Design. 1. Sources: the point-of-sale system (nightly file export) and the online store (API, paginated). 2. Ingestion: a nightly batch job with a watermark and a safety window (see the ingestion lesson). 3. Storage: bronze, silver and gold layers (medallion, see the storage lesson), in one lakehouse. 4. Transformation: SQL models with tests, run after ingestion, with a quality gate

before gold. 5. Serving: gold tables for dashboards; the customer team reads silver with masking applied. 6. Governance: catalogue entry per table with owner and freshness; personal fields classified and masked; monthly reconciliation to the ledger. 7. Failure responses: a missing file blocks the run and alerts the owner; a failed gate keeps yesterday's gold visible with a "data delayed" banner.

What was rejected and why. Streaming: not needed for daily reports and costly to operate. A second copy of the warehouse per team: duplicates cost and drifts; one lakehouse with access control is simpler.

Architecture decision records

An **architecture decision record (ADR)** is a short document that states one decision, the context, the options considered, the choice and its consequences. It stops the team from re-arguing a decision every year and explains the platform to new engineers.

MARKDOWN

```
# ADR 007: Use daily batch for retail sales reporting

Status: accepted (2026-01-15)
Context: dashboards need yesterday's sales by 07:00; no decision depends on intraday data.
Options: (a) daily batch; (b) hourly micro-batch; (c) streaming.
Decision: (a) daily batch, with a quality gate before publish.
Consequences: data is at most one day old; streaming can be added later if a requirement
appears;
we accept the cost of a nightly run and a 06:00 deadline.
```

INTERVIEW CORNER

How would you design a data platform for a company that is just starting?

Start with three requirements: what decisions the data supports, how fresh it must be, and who may see what. Choose the simplest architecture that meets them: batch ingestion into a lakehouse or warehouse, medallion layers, tested SQL models, one catalogue and access control. Write an ADR for each major choice and add streaming or extra platforms only when a measured requirement needs them.

::::key What to remember - Write measurable requirements before choosing tools; they decide the architecture. - Layers: sources, ingestion, storage, transformation, serving and consumers, with governance across all of them. - System design basics: find the bottleneck, cache with the right refresh rule, replicate and back up, and plan for each failure. - Keep the simplest design that meets the requirements, and record decisions in ADRs. ::::

Quick quiz

5 questions · tap an answer to check it

1. What should drive the choice between batch and streaming?

- **A** The newest technology
- **B** The measured freshness requirement and the cost of meeting it
- **C** The number of engineers
- **D** The colour of the dashboard

Answer: B. Freshness requirements decide whether streaming is worth its cost.

2. Why are backups needed even when data is replicated?

- **A** Replicas cannot be read
- **B** Replication copies mistakes too; backups and time travel allow recovery to an earlier state
- **C** Backups are cheaper than storage
- **D** Replicas are always out of date

Answer: B. Replication does not protect against bad data.

3. What should a design do when a quality gate fails before publishing?

- **A** Publish anyway to be safe
- **B** Keep the last good data visible and alert the owner with a delay notice
- **C** Delete the source
- **D** Stop all reporting permanently

Answer: B. Failing safe keeps trustworthy numbers visible.

4. What is the main purpose of an architecture decision record?

- **A** To replace code tests
- **B** To record a decision, its context and its consequences so it is not re-argued unnecessarily
- **C** To store passwords
- **D** To describe colours

Answer: B. ADRs preserve the reasoning behind decisions.

5. Improving a component that is not the bottleneck will:

- **A** Speed up the whole system
- **B** Change little, because the bottleneck still limits overall speed
- **C** Always cause failures
- **D** Remove the need to measure

Answer: B. The slowest component sets the overall pace.

EXERCISE **Design and record one decision**

For a company that needs a customer churn dashboard refreshed weekly and personal data protected, draw the six-layer architecture, state three measurable requirements, and write one ADR for the storage choice (warehouse, lakehouse or both), with the context, options, decision and consequences.

Show solution (worked answer)

Expected: a weekly batch pipeline, medallion layers, masking before the consumer layer, and an ADR that chooses the simplest store that meets the reporting and governance requirements, naming the trade-off accepted.

DATA ENGINEERING & CLOUD · LESSON 14

Linux and the Command Line for Data Work

Learn the terminal skills that data platforms depend on. Navigate the file system, read and search files, watch logs live, manage processes and permissions, and combine commands with pipes and redirection to answer operational questions quickly.

Gap-fill lesson | 2026 | Beginner | 120 min

By the end of this lesson you can

- Navigate and manage files and folders from the terminal
- Read, search and filter large text files and logs
- Use pipes and redirection to combine commands
- Check running processes and file permissions

Why the terminal matters for data work

Most data platforms run on Linux servers, and the fastest way to answer "is the pipeline running?" or "what went wrong last night?" is the command line. Graphical tools are useful, but a terminal session on a server is the only interface many engineers ever get. Learning a small set of commands well saves hours during an incident.

CASE STUDY **The 2 a.m. log check**

The nightly load has not finished. The on-call engineer connects to the server, finds the log, filters for errors since midnight and sees a permission error on a shared folder. Ten minutes at the terminal fix what would otherwise take an hour of clicking through a web console.

Navigating the file system

Command	Purpose	Example
`pwd`	Show the current folder	`pwd`
`ls -lh`	List files with sizes	`ls -lh /data/landing`
`cd`	Change folder	`cd /data/landing`
`find`	Search for files by name, size or age	`find /data -name "*.csv" -mtime -1`
`du -sh`	Show the size of a folder	`du -sh /data/*`
`df -h`	Show free disk space	`df -h`

`find /data -name "*.csv" -mtime -1` lists CSV files changed in the last day. It is a common first step when a job says "file not found" but the file should be there.

Reading and searching files

Terminal

```
head -n 5 daily_sales.csv          # first five lines: check the header and format
tail -n 20 pipeline.log            # last twenty lines: what happened most recently
tail -f pipeline.log               # follow a log live as new lines are written
grep -n "ERROR" pipeline.log       # lines containing ERROR, with line numbers
grep -c "ERROR" pipeline.log       # count the matching lines
wc -l daily_sales.csv              # number of lines
```

`tail -f` is the command to keep open during a run. Press Ctrl+C to stop following. `grep -n` gives the line number, which you can then open with `sed -n '120,140p' file` to read the surrounding context.

Pipes and redirection

A **pipe** (`|`) sends the output of one command into another. **Redirection** (`>` and `>>`) sends output to a file. Together they answer operational questions in one line.

Terminal

```
# How many errors happened each hour in last night's log?
grep "ERROR" pipeline.log | cut -c1-13 | sort | uniq -c

# Save today's summary of rows per file, overwriting the old file
wc -l /data/landing/*.csv > row_counts.txt

# Append a timestamp line to a run history log
echo "$(date -Iseconds) load finished" >> run_history.log
```

Symbol	Meaning
`|`	Pass the output to the next command
`>`	Write the output to a file, replacing it
`>>`	Append the output to a file
`2>&1`	Send error messages to the same place as normal output

::::tip Use `>>` for logs, `>` for results Overwriting a log with `>` by mistake erases the history of every earlier run. Use `>>` for logs that must be kept, and `>` only for output you intend to replace. ::::

Processes

A **process** is a running program. Check what is running and stop what is stuck:

Command	Purpose
`ps aux | grep python`	Show Python processes and their owners
`top` or `htop`	Live view of processes, CPU and memory use
`kill PID`	Ask a process to stop (PID is the process number)
`kill -9 PID`	Force a process to stop; use only when `kill` fails

Stop a process gently first. A forced stop can leave a file half-written, which is why an idempotent job (see the reliability lesson) is safer to restart than one that appends.

Permissions

Every file has an owner, a group and permissions for read (r), write (w) and execute (x). `ls -l` shows them:

```
-rw-r-----  1 analyst  finance  4096 Nov 11 06:00 summary.csv
```

This file is readable and writable by `analyst`, readable by the `finance` group, and hidden from everyone else. The commands that change permissions are `chmod` (mode), `chown` (owner) and `chgrp` (group).

Need	Command	Example
Let the team read a report	`chmod g+r`	`chmod g+r summary.csv`
Make a script runnable	`chmod +x`	`chmod +x load.sh`
Give a file to another owner	`chown` (needs admin rights)	`sudo chown etl:etl summary.csv`

Grant the least permission that works. Do not set everything to world-readable to fix one error; find the group that needs access and add the file to it.

::::warn Read before you delete `rm -rf` removes files without asking and without a recycle bin. Before deleting, list the matches with `ls` or `find ... -print` and check the paths. Never run a delete command with a variable that might be empty. ::::

A checklist for a server incident

1. Confirm the time and the job: `date`, then the job's log with `tail -n 100`.
2. Search for the first error: `grep -n -m 1 "ERROR"`.
3. Check resources: `df -h` (disk) and `top` (CPU and memory).
4. Check the input: `ls -lh` and `head` on the expected file.
5. Check the process: `ps aux | grep <job>`.
6. Fix, rerun, and write a line in the run history with the cause.

INTERVIEW CORNER

What commands would you use to diagnose a failing batch job on a Linux server?

Check the job's log with `tail` and `grep` for the first error, confirm the input file exists and has the expected size and header with `ls` and `head`, check disk and memory with `df` and `top`, and see whether the process is still running with `ps`. Then fix the cause and rerun, keeping a record of what happened.

::::key What to remember - The terminal is the fastest way to inspect a server and its logs. - `tail -f`, `grep`, `head` and `wc` answer most first questions about a file. - Pipes combine small

commands; `>>` keeps history, `>` replaces it. - Stop processes gently first; prefer restartable, idempotent jobs. - Check permissions before changing them, and never delete without listing first.
::::

Quick quiz

5 questions · tap an answer to check it

1. Which command follows a log file as new lines are written?

- **A** cat
- **B** head
- **C** tail -f
- **D** wc -l

Answer: C. `tail -f` keeps the file open and shows new lines as they appear.

2. What does `command > file.txt` do?

- **A** Appends to file.txt
- **B** Writes the output to file.txt, replacing its contents
- **C** Reads file.txt
- **D** Deletes file.txt

Answer: B. `>` replaces the file; `>>` appends.

3. What does the pipe in `grep ERROR log.txt | wc -l` do?

- **A** Counts the lines in the log
- **B** Counts the lines containing ERROR
- **C** Deletes the errors
- **D** Prints the errors in red

Answer: B. The pipe passes the matching lines to `wc -l`, which counts them.

4. Why prefer `kill PID` before `kill -9 PID`?

- **A** `kill -9` is slower
- **B** A gentle stop lets the program finish cleanup, reducing the chance of half-written files
- **C** `kill` deletes the process
- **D** They are identical

Answer: B. Forced stops skip cleanup.

5. Which command shows free disk space?

- **A** ls -l
- **B** ps aux
- **C** df -h
- **D** grep -c

Answer: C. `df` reports disk usage by file system.

EXERCISE

Diagnose a sample log

Create a sample log with twenty lines, including three ERROR lines and two WARNING lines (you can write it with a heredoc in the terminal). Use `grep`, `wc`, `tail` and a pipe to report the number of errors, the line number of the first error, and the last line of the log. Then write one command that appends a summary line to a separate history file.

Show solution (worked answer)

Hints: `grep -n -m 1 ERROR log.txt` gives the first error with its line number. `grep -c ERROR log.txt` counts the errors. `tail -n 1 log.txt` shows the last line. Use `>>` for the history file so earlier entries stay.

DATA ENGINEERING & CLOUD · LESSON 15

Technical Documentation, Runbooks and Production Troubleshooting for Pipelines and AI Apps

Write the documentation that keeps a data platform running when its authors are away: a README, a data dictionary and runbooks for common incidents. Then troubleshoot a live problem in a fixed order, from mitigation to root cause, and finish with a short post-incident review that adds a test or alert.

Gap-fill lesson · 2026 · Intermediate · 120 min

By the end of this lesson you can

- Write a README, a data dictionary entry and a runbook for a pipeline
- Mitigate a live problem before diagnosing its cause
- Follow a repeatable troubleshooting sequence for pipelines and AI applications
- Write a post-incident review that ends in a test, an alert or a control

Documentation is part of the system

A pipeline that only its author understands is a liability. When that person is on leave, a small failure becomes a long outage. Documentation is what lets someone else act on a problem at night, and it is cheaper to write while the system is fresh. Three documents do most of the work: a README, a data dictionary, and runbooks.

CASE STUDY **The dashboard shows yesterday's numbers**

At eight in the morning, a manager reports that the dashboard is a day behind. The engineer who built the pipeline is travelling. The runbook for "dashboard shows yesterday's numbers" lists four checks in order, names the owner of the source file, and says whom to call if the file is late. The problem is resolved by a colleague in twenty minutes.

The README

The README is the first page anyone reads. It should answer five questions in under a page:

1. **What does this do?** One paragraph, with the business purpose.
2. **What does it read and write?** Sources, destinations and refresh times.
3. **How do I run it?** Setup and the exact commands.
4. **Who owns it?** Team, named owners and how to reach them.
5. **What can go wrong?** Links to the runbooks.

MARKDOWN

```
# Sales daily load

Loads yesterday's point-of-sale files into the reporting warehouse and rebuilds the sales
dashboard tables.

- Reads: /data/landing/pos/*.csv (delivered by 05:30 by the store systems team)
- Writes: warehouse schema `gold`, tables `daily_sales`, `monthly_sales`
- Schedule: 06:00 daily (Airflow DAG `nightly_sales_close`)
- Owner: data-engineering (primary: Asha; backup: Ravi)
- Runbooks: docs/runbooks/dashboard-stale.md, docs/runbooks/load-failed.md
```

The data dictionary

A data dictionary describes every table and column: what it means, its type, its allowed values, its owner and its sensitivity. It is the document analysts reach for when a number looks strange.

Table	Column	Type	Meaning	Allowed values	Classification
daily_sales	date	date	Calendar day of the sale	Any day since 2022-01-01	Internal
daily_sales	units_sold	integer	Units sold on that day across all stores	0 or more	Internal
customers	customer_id	text	Stable customer key	Format C followed by four digits	Personal (pseudonymised in gold)

Write the meaning from the business point of view, not from the database type. "Units sold, net of returns" is a meaning; "integer" is only a type.

Runbooks

A **runbook** is a step-by-step guide for a known incident. Each runbook has the same structure, so that the reader can act under pressure:

Section	Content
Symptom	What the user or the alert sees
Impact	Who is affected and what decisions are at risk
Checks	Ordered steps, each with the command or the screen and the expected result
Fixes	The safe actions for each finding
Escalation	Who to call and when
Verification	How to confirm the problem is solved

MARKDOWN

```
# Runbook: dashboard shows yesterday's numbers

Symptom: the sales dashboard's "last refreshed" time is before 06:30.
Impact: managers see yesterday's sales; morning decisions are delayed.

Checks
1. Open the Airflow run for today. Expected: nightly_sales_close is success or running.
2. If failed, open the first failed task log. Note the error.
3. Check the landing folder: ls -lh /data/landing/pos/. Expected: one file per store,
today's date.

Fixes
- Files missing: contact the store systems team (see escalation). Do not rerun until files
arrive.
- Task failed with a transient error: clear the failed task and rerun it. The load is
idempotent.
- Quality check failed: do not override. Escalate to the data owner.

Escalation: data-engineering on-call at 07:00; the data owner after 08:00 if still
unresolved.
Verification: dashboard refresh time is after 06:30 and the row count matches the file
count.
```

::::tip Write the runbook from the last incident The best runbooks come from real incidents. After each one, add the steps that were discovered during it, and remove steps that turned out to be wrong. ::::

Production troubleshooting: the order matters

When a live system fails, people often start by looking for the cause. The better order is to limit the damage first, then find the cause, then fix it properly.

Step	Action	Why this order
1. Confirm the impact	Who is affected, and since when	Decides how urgent the response is
2. Mitigate	Rerun, roll back, switch to a safe source, or disable the feature	Stops harm while the cause is unknown
3. Check recent changes	Deployments, schema changes, new data sources, config changes	Most failures follow a change
4. Check logs and freshness	The first error, upstream freshness, row counts	Locates the failing component
5. Reproduce	Run the failing step on a copy or on a small input	Confirms the cause before a fix
6. Fix the root cause	The smallest change that removes the cause	Prevents recurrence
7. Verify	The original symptom is gone, and the checks pass	Proves the fix
8. Prevent	Add a test, an alert or a schema check	Makes the next occurrence visible early

CASE STUDY **A column that was renamed upstream**

A dashboard showed blank revenue for one region. Step 1 confirmed one region; step 2 switched the dashboard to the previous day's gold table; step 3 found that the source team had renamed a column the day before; step 4 showed the load succeeded with nulls in that column; step 5 reproduced it on a sample; step 6 mapped the new name; step 7 verified the numbers against the ledger; step 8 added a schema-change alert. Each step was short, and the sequence was the reason the incident ended quickly.

Troubleshooting an AI application

AI applications fail in new ways, so add these checks to the sequence:

- **Retrieval.** Was the right document retrieved? Look at the retrieved chunks for the failing question.

- **Model input.** Did the prompt contain what was expected (the right context, no leaked secrets, the right version)?
- **Tools.** Did each tool return a valid result? A tool error often looks like a model error.
- **Version changes.** Did the prompt, the model or the index change? Compare the regression suite with the last run.
- **Cost and latency.** A sudden rise often means a loop or a larger context than intended.

The post-incident review

A post-incident review is short and blameless. It has five parts: the timeline, the impact, the root cause, what worked and what did not, and the actions. Each action has an owner and a date.

MARKDOWN

```
# Post-incident review: revenue blank for one region (2026-11-03)

Timeline: 06:40 alert; 07:05 impact confirmed; 07:20 dashboard switched to yesterday; 09:10
fix deployed.
Impact: one region showed blank revenue for 2.5 hours; two managers used the previous day's
numbers.
Root cause: upstream renamed a column without notice; the load succeeded with nulls.
What worked: the rollback to the previous gold table; the runbook step that checked
upstream row counts.
What did not: no schema check on the source; the alert fired on the dashboard, not on the
load.
Actions:
- Add a schema-change check to the load (owner: Ravi, due 2026-11-20).
- Require notice for source schema changes in the data contract (owner: Asha, due 2026-11-
30).
```

::::warn Blameless does not mean vague "Blameless" means the review looks at the system, not at a person. It must still name the specific cause and the specific control. "Communication could be better" is not an action; "the source team posts schema changes in the channel 48 hours ahead" is. ::::

INTERVIEW CORNER

How do you troubleshoot a pipeline that is producing wrong numbers?

Confirm the impact and limit the damage first, by switching to the last good data if necessary. Check recent changes, the logs and the freshness of upstream data, and reproduce the problem on a small input. Fix the cause, verify against a trusted total, and add a test or alert so the same error is caught next time. Then write a short review with an owner for each action.

::::key What to remember - Good documentation is a README, a data dictionary and runbooks with checks, fixes and escalation. - Limit the damage first, then find the cause, then fix, verify and prevent. - Most failures follow a change; check recent changes early. - For AI applications, check retrieval, model input, tools, version changes and cost. - A post-incident review is blameless and ends with owned, dated actions. ::::

Quick quiz

5 questions · tap an answer to check it

1. What is the first priority in production troubleshooting?

- **A** Find the person who made the change
- **B** Confirm the impact and mitigate the harm
- **C** Write the post-incident review
- **D** Rebuild the whole platform

Answer: B. Stopping the harm comes before finding the cause.

2. What should a runbook contain?

- **A** Only the author's name
- **B** Symptom, impact, ordered checks, fixes, escalation and verification
- **C** A list of colours
- **D** The full source code

Answer: B. Runbooks are used under pressure, so each section must be actionable.

3. Why is a data dictionary useful?

A It makes queries faster

B It explains what each column means, its allowed values and its owner

C It replaces the database

D It stores passwords

Answer: B. Dictionaries answer "what does this number mean?"

4. In the example, what caused the blank revenue?

A A slow query

B A column renamed upstream, so the load succeeded with nulls

C A broken chart

D A missing user

Answer: B. The root cause was a silent upstream schema change.

5. Which action best prevents a repeat of a silent schema change?

A Read the logs more carefully

B Add a schema-change check that alerts before the load publishes

C Retrain the team

D Remove the dashboard

Answer: B. A preventive control makes the failure visible early.

EXERCISE

Write a runbook and a review

Write a runbook for one failure in a pipeline you know, with the symptom, impact, five ordered checks, the safe fixes and the escalation path. Then write a post-incident review for an invented incident that uses the runbook, with one owned, dated action that is a test or an alert.

Show solution (worked answer)

Hints: each check needs an expected result, so that the reader knows when to move on. The action must be something you can test: a schema comparison, a freshness alert, or a reconciliation.